hexens x

Security Review Report for MetaLex

August 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - CREATE2 salt squatting allows offering signatures to be reused and payments to be redirected
    - Frozen CyberScrip positions can be moved to unfrozen accounts through certificate conversion
    - Stale authority signatures can be replayed to block an investor's allocation
    - Permissionless RoundManager deployment allows LeXcheX minting without authority attestation
    - Repeated partial fills let buyers underpay for secondary offers
    - Caller-selected dev mode lets one ZKPassport qualify multiple wallets
    - Cross-certificate vault share growth can disable scripification and break share-bearing certificate conversions
    - convertScripToCert can cause newly acquired restricted securities to inherit an old holding period
    - Seller extensionData is copied to buyer, inheriting Rule 144 tacking anchor
    - Standalone agreement signatures can be reused past the intended expiry
    - Upgrading certificate printers freezes custody transfers of legacy certificates
    - Upgrading a live CyberScrip can block complete redemption
    - Stale endorsements allow former owners to reclaim legal title
    - Mint callback allows recertification approval reuse

- Permissionless agreement creation can block secondary trades
- Country policy updates do not invalidate cached zkPassport eligibility
- Certificate unit movements do not preserve partly paid balances
- CyberScrip applies transfer hooks to mints
- Round authorization can be reused through a factory with weaker metadata validation
- Voided certificates can be used in secondary trades
- Issuer authorization does not bind investor-supplied agreement terms
- Buyer-selected DealManager custody permits transfers outside the secondary settlement flow
- Single admin can bypass the two-key kill switch
- Look-through tally can be corrupted if the badge is wired late
- Auto-mint transfer failure blocks large investor allocations
- Burning a state credential makes a blocked buyer appear non-U.S.
- Buyer revocation does not invalidate an existing settlement authorization
- voidContractFor incorrectly voids zero-expiry agreements
- Zero adminMultisig Can Lock Secondary Trade Settlements
- Ownership transfer does not revoke the previous owner
- Certificate Metadata Can Falsely Report the Holder’s Balance
- revokeDeal leaves cancelled certificates active
- Cross-holder de-scripification leaves ghost holders in the holder count
- A shared delegate's signature can be replayed for multiple parties
- timestampToDate Returns Incorrect Calendar Dates
- Fee is determined at execution, allowing fee changes to affect existing trades
- acceptOwnership() emits an incorrect RoleUpdated event
- voidCert allows non-existent certificates to be marked as void
- Core component setters emit no events

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Metalex protocol, a platform for trusted parties to trade or tokenize real-world assets (RWA).

Our security assessment was a full review of the code, spanning a total of 4 weeks.

During our review, we identified multiple high severity vulnerability, which could have resulted in loss of assets.

We also identified some minor severity vulnerabilities and code optimizations.

All reported issues were fixed or acknowledged by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

Review Led by

Kasper Zwijsen, Head of Audits

Scope

The analyzed resources are located on:

https://github.com/MetaLex-Tech/cybercorps-contracts/tree/bbcd3e5a86df90d63f3324db47fe625eb1a4b12d

The issues described in this report were fixed in the following commit:

https://github.com/MetaLex-Tech/cybercorps-contracts/tree/2bf6a3bd62fdcee93c4ac5e829104bdb4a2b2131

Changelog

23 July 2026	Audit start
24 August 2026	Initial report
20 September 2026	Revision received
1 October 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

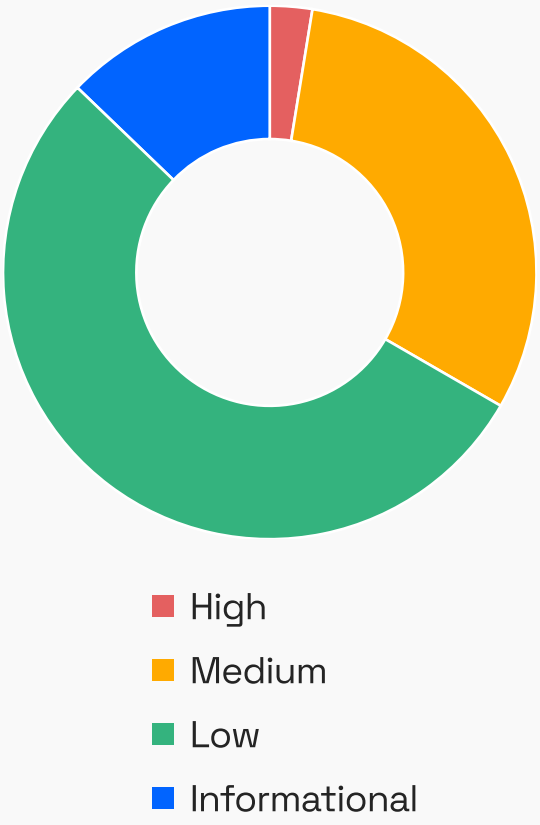
- **Issue Symbolic Codes**

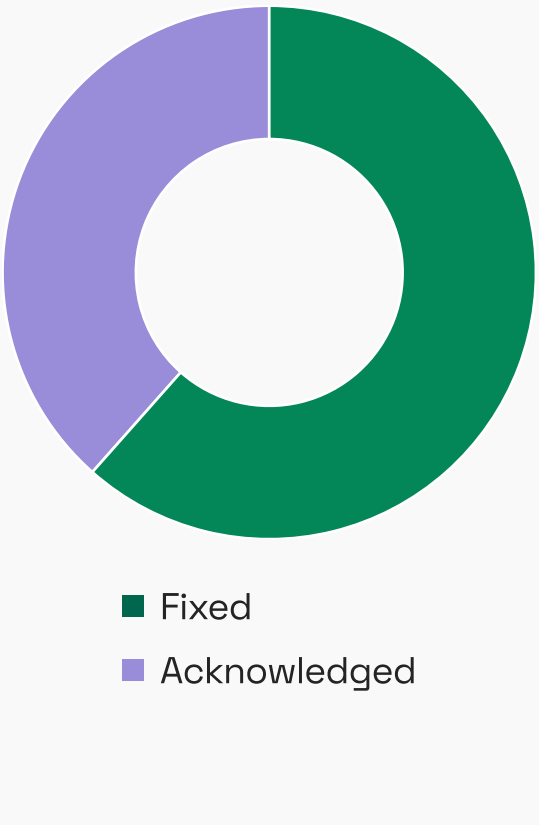
Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	1
Medium	12
Low	21
Informational	5
Total:	**39**





6. Weaknesses

This section contains the list of discovered weaknesses.

MTLX1-41 | CREATE2 salt squatting allows offering signatures to be reused and payments to be redirected

Fixed

Severity:	High	Probability:	Likely	Impact:	High

Path:

metalex-jul-26/src/PumpCorpFactory.sol:deployCyberCorp#L251-L341

Description:

The combined deployment functions create a **CyberCorp** and its managers before creating a signed fundraising round or primary offer. They are meant to deploy the company at the addresses approved by the officer and then use the officer's signature only with that company and its intended payout settings. The same flow is also exposed by **CyberCorpFactory**.

Normally, the deployment and offering authorization should describe one complete setup. This includes the company's authority, officers, payout address, and the important offering terms. Nobody else should be able to deploy different contracts at the approved addresses and still use the signature.

However, the raw **deployCyberCorp** function is public, and its CREATE2 addresses depend only on a caller-supplied salt. An attacker who sees a pending call to **deployCyberCorpAndCreateRoundFor** can use its salt first and deploy the exact predicted company and manager addresses. The attacker can choose their own officer and **companyPayable**, while the legitimate deployment later reverts because those addresses are already occupied.

The round signature is still valid on the attacker's **RoundManager**. As shown in **metalex-jul-26/src/storage/RoundManagerStorage.sol (L124-145)**, it binds the manager through the EIP-712 domain and includes the predicted company address, but not the company's authority, controlling officer, or payout address. **metalex-jul-26/src/RoundManager.sol:createRound (L199-248)** also accepts the signed officer from the supplied draft without linking it to an authorized company deployment.

The primary-offer path has the same issue and additionally leaves the payment token, amount, certificates, and conditions outside the signed agreement data.

An attack can happen as follows:

1. An officer signs a round or primary offer and broadcasts a combined deployment using a fresh salt.
2. An attacker copies the salt and calls the public raw deployment first, setting themselves as officer and using their own **companyPayable**.
3. The attacker creates the offering on the squatted manager using the officer's observed signature. Verification passes because the signed company and manager addresses did not change.
4. Investors pay into the apparently officer-approved offering. On allocation or finalization, the escrow sends the payment to the attacker's **companyPayable**.

```
function deployCyberCorp(
    bytes32 salt,
    string memory companyName,
    string memory companyType,
    string memory companyJurisdiction,
    string memory companyContactDetails,
    string memory defaultDisputeResolution,
    address _companyPayable,
    CompanyOfficer memory _officer
)
    public
    returns (
        address cyberCorpAddress,
        address authAddress,
        address issuanceManagerAddress,
        address dealManagerAddress,
        address roundManagerAddress
    )
{
    if (salt == bytes32(0)) revert InvalidSalt();

    // Deploy BorgAuth with CREATE2 with new param address owner
    bytes memory authBytecode = type(BorgAuth).creationCode;
    bytes32 authSalt = keccak256(abi.encodePacked("auth", salt));
    authAddress = Create2.deploy(
        0,
        authSalt,
        abi.encodePacked(authBytecode, abi.encode(address(this)))
    );

    // Initialize BorgAuth
    // BorgAuth(authAddress).initialize();
    BorgAuth(authAddress).updateRole(_officer.eoa, 200);
```

```
issuanceManagerAddress = IIssuanceManagerFactory(issuanceManagerFactory)
    .deployIssuanceManager(salt);

cyberCorpAddress = ICyberCorpSingleFactory(cyberCorpSingleFactory)
    .deployCyberCorpSingle(salt);

// Initialize CyberCorp
ICyberCorp(cyberCorpAddress).initialize(
    authAddress,
    companyName,
    companyType,
    companyJurisdiction,
    companyContactDetails,
    defaultDisputeResolution,
    issuanceManagerAddress,
    _companyPayable,
    _officer,
    cyberCorpSingleFactory,
    address(0)
);

BorgAuth(authAddress).updateRole(cyberCorpAddress, 200);
//deploy deal manager
dealManagerAddress = IDealManagerFactory(dealManagerFactory)
    .deployDealManager(salt);
ICyberCorp(cyberCorpAddress).setDealManager(dealManagerAddress);
// Initialize IssuanceManager
IIssuanceManager(issuanceManagerAddress).initialize(
    authAddress,
    cyberCorpAddress,
    uriBuilder,
    issuanceManagerFactory
);

// Initialize DealManager
IDealManager(dealManagerAddress).initialize(
    authAddress,
    cyberCorpAddress,
    registryAddress,
    issuanceManagerAddress,
    dealManagerFactory
);

// Deploy and initialize RoundManager
roundManagerAddress = deployAndInitializeRoundManager(salt, cyberCorpAddress);
```

```
        // Authorize peripheral contracts for the cyber corp. It is ok to do it here on behalf of the corp
        // because the corp has just been created by us.
        // In contrast, if any of the peripheral contract is being retrofitted to an existing corp,
        // they would have to authorize it themselves for security reasons.

        // Set RoundManager on the corp
        ICyberCorp(cyberCorpAddress).setRoundManager(roundManagerAddress);

        BorgAuth(authAddress).updateRole(issuanceManagerAddress, 99);
        BorgAuth(authAddress).updateRole(dealManagerAddress, 99);
        BorgAuth(authAddress).updateRole(roundManagerAddress, 99);

        emit CyberCorpDeployed(
            cyberCorpAddress,
            authAddress,
            issuanceManagerAddress,
            dealManagerAddress,
            companyName,
            companyType,
            companyContactDetails,
            companyJurisdiction,
            defaultDisputeResolution,
            _companyPayable
        );
    }
```

Remediation:

Make the raw corporate deployment functions internal and restrict the component factories to authorized callers, or derive and consume the CREATE2 salt from a signed deployment authorization. The signed data should also bind the corporate authority, officers, payout address, and all security-relevant offering and escrow terms.

MTLX1-1 | Frozen CyberScrip positions can be moved to unfrozen accounts through certificate conversion

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Description:

CyberScrip applies account-freeze checks only when the recipient is not the zero address. Consequently, burns performed during certificate conversion do not check whether the source account is frozen.

```
    function _update(address from, address to, uint256 amount) internal virtual override {
        ...
        // Enforce freeze checks for normal transfers (not mint/burn)
        if (to != address(0)) {
            if (s.canFreeze) {
                if (s.frozen[from]) revert AccountFrozen(from);
                if (s.frozen[to]) revert AccountFrozen(to);
            }
            // Enforce transfer restriction hooks
            uint256 length = s.transferRestrictionHooks.length;
            for (uint256 i = 0; i < length; i++) {
                (bool allowed, string memory reason) = s.transferRestrictionHooks[i].checkTransferRestriction(from,
to, amount, "");
                if (!allowed) revert RestrictedTransfer(reason);
            }
        }
        ...
```

```
    function burnFrom(address account, uint256 amount) public virtual onlyIssuanceManager {
        super._burn(account, amount);
    }
```

convertScripToCert uses this burn path without first checking whether account is frozen.

```
    function executeConvertScripToCert(
        address certAddress,
        uint256 amount,
        address account,
        bytes4 convertSelector
```

```
) external {
    ...
    ICyberScrip(scripifiedCert).burnFrom(account, amount);

    if (selection.foundActive) {
        CertificateDetails memory activeDetails = certificate
            .getActiveCertificateDetails(selection.activeTokenId);
        activeDetails.unitsRepresented =
            activeDetails.unitsRepresented +
            units;
        certificate.updateCertificateDetails(
            selection.activeTokenId,
            activeDetails
        );
    ...
```

The holder can subsequently call **scripifyCert** and select an arbitrary unfrozen address as the mint target. The function verifies that account owns the certificate, but does not check whether that account is frozen.

```
function executeScripifyCert(
    address certAddress,
    uint256 id,
    uint256 amount,
    address target,
    address account
) external {
    ...
    ILedgerEntryToken certificate = ILedgerEntryToken(certAddress);
    if (certificate.isVoided(id)) revert CertificateVoided();
    if (certificate.legalOwnerOf(id) != account) revert NotLegalOwner();

    address toSend = target;
    if (toSend == address(0)) toSend = account;

    ...
    ICyberScrip(scripifiedCert).mint(toSend, scripAmount);
    ...
}
```

A frozen holder can therefore convert its scrip into certificate units and reissue the same position to an unfrozen address. The recipient can transfer or sell the reissued scrip while the original

account remains frozen. Supply and backing remain unchanged, but the freeze no longer immobilizes the position.

Remediation:

- Reject **convertScripToCert** when the source scrip account is frozen.
- Reject **scripifyCert** when the certificate's legal owner is frozen, regardless of the mint target.

MTLX1-17 | Stale authority signatures can be replayed to block an investor's allocation

Acknowledged

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

src/creds/lexchexMinter.sol

Description:

requestRenewal() accepts an authority signature without requiring it to be unused, expired, or bound to a specific **tokenId**. The signature is also reusable because it is never consumed.

Authority signatures are publicly visible in the calldata of the original **requestMint** transaction. As a result, anyone can take an old signature and replay it through **requestRenewal()** to set a holder's credential expiry back into the past.

If the affected credential is required by **LexChexCondition**, the investor can no longer be allocated even though they have already deposited their funds.

Example

1. MetaLeX issues Alice a credential expiring on day 365. The authority signature becomes public in the **requestMint** transaction.
2. On day 360, MetaLeX renews Alice's credential until day 725. Alice deposits 50,000 USDC into a round.
3. An attacker retrieves the old day-0 authorization through **requestRenewal()**, which has a expiry set at day 365 (from 725).
4. After day 365, the attacker can replay this signature, immediately making Alice's credential invalid.
5. The GP attempts to allocate Alice, but her credential has expired, causing **LexChexCondition** to fail and **allocate()** to revert.
6. Even if MetaLeX re-credentials Alice, the attacker can replay the same old authorization again because it is never consumed or expired.

The attacker can repeatedly prevent the victim from receiving their allocation.

Remediation:

Require renewal authorizations to be single-use, unexpired, and bound to the specific **tokenId**. Additionally, only allow renewals to move the credential expiry forward.

This prevents replay attacks, expiry rollbacks, and cross-token reuse.

MTLX1-3 | Permissionless RoundManager deployment allows LeXcheX minting without authority attestation

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Description:

CyberCorpFactory.deployAndInitializeRoundManager is public and does not validate its caller or the provenance of **cyberCorpAddress**. It initializes a **RoundManager** with values returned by that address and grants the manager **OWNER_ROLE** in the shared **LeXcheX** authority.

```
    function deployAndInitializeRoundManager(bytes32 salt, address cyberCorpAddress) public returns
(address) {
        if (ICyberCorp(cyberCorpAddress).roundManager() != address(0)) {
            revert RoundManagerAlreadyExists();
        }

        address roundManagerAddress =
IRoundManagerFactory(roundManagerFactory).deployRoundManager(salt);

        // Initialize RoundManager
        IRoundManagerInit(roundManagerAddress).initialize(
            address(BorgAuthACL(cyberCorpAddress).AUTH()),
            cyberCorpAddress,
            registryAddress,
            ICyberCorpLocal(cyberCorpAddress).issuanceManager(),
            roundManagerFactory
        );

        // Add newly created RoundManager as OWNER in LeXcheX AUTH
        if (lexchexAuth != address(0)) {
            BorgAuth(lexchexAuth).updateRole(
                roundManagerAddress,
                BorgAuth(lexchexAuth).OWNER_ROLE()
            );
        }

        emit RoundManagerDeployed(
            cyberCorpAddress,
            roundManagerAddress
        );
```

```
        return roundManagerAddress;
    }
```

An attacker-controlled contract can provide its own local authority and issuance manager address, allowing the attacker to manage the resulting **RoundManager**. Because **BorgAuth** roles are hierarchical, the manager's **OWNER_ROLE** satisfies the **onlyAdmin** restriction on requestMintFor.

```
    function requestMintFor(
        MintRequest memory request,
        bytes32 _templateId,
        uint256 _salt,
        string[] memory _globalValues,
        address[] memory _parties,
        string[][] memory _partyValues,
        bytes memory agreementSignature
    ) external onlyAdmin returns (bytes32 agreementId, uint256 tokenId) {
        ...
        acc.agreementId = agreementId;
        tokenId = LeXcheX(lexchex).mint(request.owner, acc);
        ICyberAgreementRegistry(dealRegistry).finalizeContract(agreementId);

        emit MintRequested(request.owner, request.mintPrice, agreementId);
        emit MintCompleted(request.owner, tokenId, agreementId);
    }
```

Unlike the standard minting path, **requestMintFor** does not require an authority signature over the accreditation data. The credential owner must still provide a valid agreement signature, but that signature only confirms participation in the agreement.

The automatic minting branch in allocate checks whether **escrow.counterParty** already holds a valid credential, while **requestMintFor** mints the credential to **request.owner**. These addresses are not required to match, so the requested recipient is not bound to the investor whose payment triggered the mint.

```
    function allocate(
        LexScrowStorage.LexScrowData storage ls,
        bytes32 agreementId,
        uint256 allocatedAmount
    )
    ...
```

```
        if (!ILexChex(getLexChex()).hasValidLexCheX(escrow.counterParty)) {
            if (IRoundManagerFactory(getUpgradeFactory()).isWhitelistedToken(round.paymentToken)) {
                if (usedAmount1e18 >= 200000 * 1e18 && eoi.naturalPerson) {
                    (, tokenId) = ILexChexMinter(getLexChexMinter()).requestMintFor(eoi.lexchexDetails.request,
eoi.lexchexDetails.templateId, eoi.lexchexDetails.salt, eoi.lexchexDetails.globalValues,
eoi.lexchexDetails.parties, eoi.lexchexDetails.partyValues, eoi.lexchexDetails.agreementSignature);
                }
                if (usedAmount1e18 >= 1000000 * 1e18 && !eoi.naturalPerson) {
                    (, tokenId) = ILexChexMinter(getLexChexMinter()).requestMintFor(eoi.lexchexDetails.request,
eoi.lexchexDetails.templateId, eoi.lexchexDetails.salt, eoi.lexchexDetails.globalValues,
eoi.lexchexDetails.parties, eoi.lexchexDetails.partyValues, eoi.lexchexDetails.agreementSignature);
                }
            }
        }
```

The attacker can create a zero-certificate FCFS round and use a whitelisted token to satisfy the automatic minting threshold. Settlement transfers the supplied amount, less the configured fee, to the attacker-controlled **companyPayable** address. The resulting credential is not scoped to the issuing corp and can satisfy other flows that use the same **LeXcheX** instance and rely on **hasValidLexCheX**.

Remediation:

- Restrict the migration entry point to recognized legacy corps and validate the supplied corp before assigning any shared role.
- Replace shared **OWNER_ROLE** or **ADMIN_ROLE** grants with a narrowly scoped, round-bound minting permission.
- Require independent authority attestation for accreditation data used by automatic minting.
- Bind the credential owner, payment token, template, and request terms to the corresponding counterparty and round.

MTLX1-32 | Repeated partial fills let buyers underpay for secondary offers

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

metalex-jul-26/src/storage/SecondaryTradeStorage.sol:_acceptOffer#L319-L495

Description:

The **_acceptOffer** function lets an eligible buyer accept all or part of a secondary offer. The normal call path is **DealManager.acceptOffer** -> **SecondaryTradeStorage.acceptOffer** -> **_acceptOffer**; the resulting escrow is later settled through **finalizeSecondaryTradeAgreement**, which pays the seller and transfers the securities.

Partial fills should keep the seller's posted price regardless of how the buyer splits the order. In other words, after some units have been accepted, the total payment collected should match the cumulative pro-rata price for those units, with any final remainder charged on the last fill.

Instead, each non-final fill is priced on its own and rounded down:

```
uint256 partialConsideration = params.units == remainingUnits
    ? offer.consideration - offer.paymentAccepted
    : offer.consideration * params.units / offer.units;
```

Because earlier rounding is not included in the next fill, a buyer can repeat small fills and discard a fraction each time. The final fill inherits all of that rounding debt, but the buyer can simply leave it behind. The zero-payment and minimum-trade checks do not help when every discounted fill still costs at least one raw token unit.

For example, consider a SELL offer for 11 shares priced at 21 raw units of a 0-decimal payment token:

1. The attacker submits ten separate fills of one share each, using the usual signatures and eligibility checks.
2. Every fill costs **floor(21 / 11) = 1** token unit, so **_acceptOffer** escrows only 10 units for 10 shares. A cumulative calculation would have collected 19 units at this point.
3. The remaining share now costs 11 units because the final-fill branch includes all prior rounding debt. The attacker does not buy it.
4. The ten accepted escrows can still be finalized, even if the seller cancels the remaining share. The attacker therefore receives 10 shares for 10 token units instead of their roughly 19.09-unit pro-rata price.

Remediation:

Calculate each fill from the cumulative target payment, for example **mulDiv(offer.consideration, offer.unitsAccepted + params.units, offer.units) - offer.paymentAccepted**, while keeping the exact remainder for the final fill. This makes the total paid independent of how the offer is split.

MTLX1-33 | Caller-selected dev mode lets one ZKPassport qualify multiple wallets

Acknowledged

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

metalex-jul-26/src/libs/conditions/NonUSNationalityCondition.sol:submitProof#L108-L153

Description:

The **submitProof** function verifies a ZKPassport proof and caches the caller's eligibility. It checks the proof's scope, wallet, chain, nationality, and sanctions status, then stores an expiry for both the wallet and the verifier-provided **uniqueIdentifier**. Later, **checkCondition** uses the wallet expiry to decide whether that wallet may take part in a gated round.

The **uniqueIdentifier** should be stable for the same passport and service scope. This lets the contract reject a second wallet when the same passport already has an active proof, preserving the intended one-passport/one-wallet rule.

However, **submitProof** forwards the caller-supplied **params.serviceConfig** without rejecting **devMode**. The default verifier allows salted nullifiers in dev mode, and the proof creator chooses the nullifier secret. Changing this secret gives the same real passport a new **uniqueIdentifier**, so the following check never sees the earlier proof:

```
(bool verified, bytes32 uniqueIdentifier, IZKPassportHelper helper) = verifier.verify(params);

if (uniqueIdentifierExpiry[uniqueIdentifier] >= block.timestamp) revert ProofAlreadyUsed();
```

An eligible passport holder can therefore qualify any number of wallets:

1. They create a dev-mode proof for wallet A using salt **s1**. The verifier returns **U1**, and **submitProof** caches eligibility for wallet A.
2. They create another proof from the same passport for wallet B using salt **s2**. The verifier returns a different identifier, **U2**, so **ProofAlreadyUsed** does not trigger.
3. Wallets A and B both pass **checkCondition** and can proceed through allocation and security issuance. The process can be repeated with more wallets and salts.

This uses genuine passport proofs and does not require a compromised verifier or a forged identity.

```
    function submitProof(
        ProofVerificationParams calldata params,
        bool isIDCard
    ) external {
        (bool verified, bytes32 uniqueIdentifier, IZKPassportHelper helper) = verifier.verify(params);
        if (!verified || address(helper) == address(0)) revert InvalidProof();

        if (uniqueIdentifierExpiry[uniqueIdentifier] >= block.timestamp) revert ProofAlreadyUsed();

        if (
            !helper.verifyScopes(
                params.proofVerificationData.publicInputs,
                expectedDomain,
                expectedScope
            )
        ) {
            revert InvalidScope();
        }

        BoundData memory boundData = helper.getBoundData(params.committedInputs);
        if (boundData.senderAddress != msg.sender) revert InvalidBoundSender();
        if (boundData.chainId != block.chainid) revert InvalidBoundChainId();

        if(!helper.isNationalityOut(excludedCountries, params.committedInputs)) revert
USAOrSanctionedCountriesNotAllowed();

        uint256 proofTimestamp = helper.getProofTimestamp(
            params.proofVerificationData.publicInputs
        );

        // Check against the sanctioned watchlist at the time of the proof
        helper.enforceSanctionsRoot(
            proofTimestamp,
            false,
            params.committedInputs
        );

        uint256 validityPeriod = params.serviceConfig.validityPeriodInSeconds;
        if (validityPeriod > maxValidityPeriod) revert MaxValidityPeriodExceeded();
        uint256 expiresAt = proofTimestamp + validityPeriod;
        if (expiresAt < block.timestamp) revert ProofExpired();
        if (expiresAt > block.timestamp + maxValidityPeriod) revert MaxValidityPeriodExceeded();

        proofExpiry[msg.sender] = expiresAt;
```

```
        uniqueIdentifierExpiry[uniqueIdentifier] = expiresAt;
        emit ProofSubmitted(msg.sender, expiresAt);
    }
```

Remediation:

Reject proofs where **params.serviceConfig.devMode** is true and require the verifier-authenticated nullifier type to be **NON_SALTED_NULLIFIER**. Ideally, construct the service configuration in the contract instead of accepting its security-sensitive fields from the caller.

MTLX1-5 | Cross-certificate vault share growth can disable scripification and break share-bearing certificate conversions

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Description:

Scripified certificate units sit in a shared ERC-4626-style vault per printer (**totalAssetsWad, totalNominalShares**). Deposits mint nominal shares to the certificate that funded them. Redemptions against that certificate's own claim burn shares and assets together. Conversions above the selected certificate's claim instead remove assets only:

```
function executeConvertScripToCert(
    address certAddress,
    uint256 amount,
    address account,
    bytes4 convertSelector
) external {
...
    if (selection.foundActive) {
        uint256 claimWad = _assetsOfVaultPosition(
            certAddress,
            selection.activeTokenId
        );
        uint256 fromVaultWad = units < claimWad ? units : claimWad;
        uint256 redeemedWad;
        if (fromVaultWad > 0) {
            redeemedWad = _redeemVaultForCert(
                certAddress,
                selection.activeTokenId,
                fromVaultWad
            );
        }
        if (units > redeemedWad) {
            _withdrawVaultAssets(certAddress, units - redeemedWad);
        }
    }
...
```

Asset-only withdrawal is intentional for fungible scrip (burned scrip keeps pool backing consistent). The defect is that shares minted on deposit are not retired on that path.

```
    function _withdrawVaultAssets(
        address certAddress,
        uint256 assetsOutWad
    ) internal {
        CertScripUnitPool storage pool = issuanceManagerStorage().certScripUnitPools[
            certAddress
        ];
        if (pool.totalAssetsWad == 0) revert EmptyVault();
        if (assetsOutWad > pool.totalAssetsWad) {
            revert VaultWithdrawalExceedsAssets();
        }

        pool.totalAssetsWad -= assetsOutWad;
        if (pool.totalAssetsWad == 0) {
            _resetVaultPositions(certAddress);
        }
    }
```

Deposit and convert also pick certificates differently. **scripifyCert** takes an explicit token id. **convertScripToCert** takes only the printer (**certAddress**). The recipient is the first non-voided legal-owned certificate:

```
function _selectFirstLegalOwnedToken(address certAddress, address owner)
    internal view returns (RecertSelection memory selection)
{
    ...
    for (uint256 i = 0; i < ownedBalance; i++) {
        uint256 tokenId = certificate.tokenOfLegalOwnerByIndex(owner, i);
        if (certificate.isVoided(tokenId)) continue;
        selection.foundActive = true;
        selection.activeTokenId = tokenId;
        return selection;
    }
}
```

Secondary settlement mints a fresh certificate per acquisition. One address can therefore hold two lots on the same printer through ordinary trading (for example two partial fills).

An attacker can then:

1. Keep one certificate as a share-less conversion sink (the lot that sorts first in **_selectFirstLegalOwnedToken**).

2. Scripify units from the second certificate via the explicit id argument, minting vault shares onto that certificate.
3. Convert the scrip. Because the sink's vault claim is zero, the full amount goes through **_withdrawVaultAssets**: assets fall, but the shares minted in step 2 remain.
4. Repeat. Pool assets can return to a stable nonzero floor (for example one unit left by any holder), while **totalNominalShares** multiplies each cycle.

Once **totalNominalShares** is large enough, checked arithmetic reverts on:

- further deposits (**assets * totalNominalShares / totalAssetsWad in _depositCertScripUnits**)
- claim and detail reads that multiply position shares by pool assets (**_assetsOfVaultPosition** and callers such as **getCertificateDetails** for share-bearing lots)
- conversions that need those claim calculations.

Remediation:

- Bind conversion to an explicit certificate token ID and reject any amount exceeding that certificate's vault claim, or otherwise require conversion to consume shares from the same position that funded the units.
- If socialized withdrawals remain supported, redesign the accounting so both global and per-certificate shares are adjusted consistently. Do not reduce only **totalNominalShares**.

MTLX1-8 | convertScripToCert can cause newly acquired restricted securities to inherit an old holding period

Acknowledged

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

src/storage/IssuanceManagerStorage.sol

Description:

convertScripToCert can cause newly acquired restricted securities to inherit an old holding period.

SEC Rule 144 provides a safe harbor for reselling certain restricted securities, subject to requirements such as a minimum holding period. In this system, **acquisitionTimestamp** is used to track that period, so each lot should have its own timestamp.

When scrip is converted back into a certificate, the new units should therefore receive a fresh **acquisitionTimestamp**. However, if the user already owns a certificate in the same series, **executeConvertScripToCert** takes the merge branch and adds the new units to the existing lot. The existing **acquisitionTimestamp** is left unchanged, causing the newly acquired units to inherit the old holding period.

This allows a user with an near/already-seasoned lot to acquire restricted scrip and make the new units appear seasoned when converted back into a certificate.

This also creates a discount arbitrage. Restricted securities are typically worth less because the buyer must wait for the holding period to expire before resale. By inheriting an old timestamp, a buyer can purchase the restricted scrip at a discount and then resell the resulting securities at the unrestricted price without waiting for the required holding period.

Example 1: Bypassing the holding period

Assume the required holding period is 12 months.

1. Alice acquires dust amountof a certificate.
2. Alice holds it for more than 11 months, so the lot is almost seasoned.
3. Bob acquires 1e18 restricted shares and has held them for only one month.
4. Bob scripifies those shares and sells the scrip to Alice, because the scrip has not satisfied the holding period, Bob has to sell it at a discount.
5. Alice calls **convertScripToCert**.
6. Alice already owns a certificate for the same series, so the conversion takes the merge branch.
7. The 1e18 newly acquired units are added to Alice's existing lot.

8. The old **acquisitionTimestamp** remains unchanged.
9. Alice can now wait 1 month and pass **HoldingPeriodCondition** for the entire position.

Example 2: Discount arbitrage

Assume the required holding period is 12 months.

1. An unrestricted security is worth $100/unit, while a security with 11 months remaining on its holding period trades at $90/unit because the buyer must wait for liquidity.
2. Bob owns 1,000 restricted units that have only been held for one month.
3. Bob scripifies those units and sells the resulting scrip to Alice for $90,000, accepting the discount because the units are still restricted.
4. Alice already owns a tiny certificate lot that has been held for close to 12 months.
5. Alice calls **convertScripToCert** with Bob's 1,000 scrip.
6. Because Alice already owns a certificate for the same series, the conversion takes the merge branch.
7. The 1,000 newly acquired units are added to Alice's almost seasoned lot.
8. The old **acquisitionTimestamp** remains unchanged, so the new units are treated as though they were acquired more than 12 months ago.
9. Alice can now sell the 1,000 units at the unrestricted price of $100,000, without waiting for their own 12-month holding period.

The $10,000 discount exists specifically to compensate the buyer for waiting 11 more months for liquidity. Because Alice can inherit the old lot's holding period, she captures this discount without actually serving the required holding period.

This creates a direct arbitrage opportunity and undermines the purpose of the Rule 144 holding-period restriction.

Root Cause

The problematic branch is inside **executeConvertScripToCert**:

```
if (selection.foundActive) {
    activeDetails.unitsRepresented += units;
    certificate.updateCertificateDetails(
        selection.activeTokenId,
        activeDetails
    );
}
```

updateCertificateDetails updates the number of units but does not create a new lot or update the **acquisitionTimestamp**. The newly acquired units therefore inherit the existing lot's timestamp.

cyberTrade spec v 4.1 specification explicitly distinguishes between resetting and tacking acquisition dates:

*“**acquisitionDate** on the buyer's new Ledger Entry Token … **resets for Rule 144 and Reg S; tacking-eligible carry-over for §4(a)(7) and §4(a)(1½) where the buyer would inherit a Rule 144 tacking position under §144(d)(3)`”*

MTLX1-9 | Seller extensionData is copied to buyer, inheriting Rule 144 tacking anchor

Acknowledged

Severity:	Probability:	Impact:
Medium	Likely	Medium

Description:

executeSecondaryTransfer correctly gives the buyer's new certificate a fresh **acquisitionTimestamp**, but it also copies the seller's entire **extensionData**.

This includes **tackedFromAcquisitionDate**, the Rule 144(d)(3) tacking anchor. **HoldingPeriodCondition** uses the earlier of the buyer's acquisition timestamp and this anchor, allowing the buyer to inherit the seller's holding period and potentially resell immediately.

once an anchor is attached to a lot, every subsequent buyer inherits it.

Root Cause

The buyer's certificate is created using the seller's extension data:

```
CertificateDetails memory buyerDetails = CertificateDetails({
    ...
    unitsRepresented: units,
    legalDetails: sellerDetails.legalDetails,
    extensionData: sellerDetails.extensionData
});
```

The specification explicitly states:

“acquisitionDate on the buyer's new Ledger Entry Token … resets for Rule 144 and Reg S; tacking-eligible carry-over for §4(a)(7) and §4(a)(1½) where the buyer would inherit a Rule 144 tacking position under §144(d)(3)”

However, **executeSecondaryTransfer** copies the tacking anchor on every secondary-transfer pathway. **exemptionPathway** is decoded but not used to determine whether tacking is permitted.

The specification also states that **HoldingPeriodCondition**:

“applies the earlier date where Rule 144(d)(3) tacking is asserted”

The copied anchor is an assertion originally made for the seller, not the new buyer. The buyer's entitlement is never re-evaluated.

Impact

A seller with a legitimately asserted tacking anchor can sell the lot to a buyer who immediately receives the same anchor.

For example:

1. Seller's lot is only 10 days old.
2. Issuer legitimately sets a tacking anchor from 400 days ago.
3. Seller transfers the lot to Alice.
4. Alice receives a fresh base **acquisitionTimestamp**.
5. The seller's 400-day **tackedFromAcquisitionDate** is copied to Alice.
6. **HoldingPeriodCondition** uses the older anchor.
7. Alice can immediately resell the newly acquired securities.

The anchor can then propagate from Alice to the next buyer and so on.

```
function test_PoC_BuyerInheritsSellersTackingAnchorAndResellsImmediately() public {

ILedgerEntryToken p = _printer("FUNDA");
uint256 sellerLot = _mintWithFundData(p, seller, UNITS);

vm.warp(block.timestamp + 10 days);
assertFalse(_canSell(p, sellerLot, seller, 1));

// Issuer legitimately asserts a 400-day Rule 144(d)(3) anchor for the seller.
uint64 anchor = uint64(block.timestamp - 400 days);
LedgerEntryToken(address(p)).updateCertificateTackedFromAcquisitionDate(
    sellerLot,
    anchor
);
assertTrue(_canSell(p, sellerLot, seller, 2));

// Buyer acquires the lot through an ordinary RULE_144 secondary trade.
uint256 buyerLot = _tradeAll(p, sellerLot, 3);

// Buyer's base acquisition timestamp is correctly reset.
assertEq(
    p.acquisitionTimestamp(buyerLot),
    uint64(block.timestamp)
);

// But the seller's tacking anchor is copied through extensionData.
bytes memory buyerExt =
    p.getActiveCertificateDetails(buyerLot).extensionData;

assertEq(
```

```
    ext.tackedFromAcquisitionDate(buyerExt),
    anchor
);

// The buyer can therefore resell immediately.
assertTrue(_canSell(p, buyerLot, buyer, 4));
}
```

Remediation:

Conversion must mint a fresh lot. If that mint populates **extensionData** from a prior lot, clear **tackedFromAcquisitionDate** first otherwise the fix restores the base clock while leaving the effective anchor inherited, and the bypass survives.

MTLX1-39 | Standalone agreement signatures can be reused past the intended expiry

Acknowledged

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

metalex-jul-26/src/CyberAgreementRegistry.sol:createStandaloneContractAndSignFor#L359-L409

Description:

The standalone creation helpers let a party create an agreement and submit its signature in one transaction. The helper creates a deterministic agreement through **createContract**, then passes the signature to **signContractFor**. These functions apply to any party creating or signing a standalone agreement.

The expiry should limit how long the parties can sign the agreement. A signature prepared for an agreement that expires at time **T** should therefore not be usable to finalize that agreement after T.

However, **createContract** does not include **expiry** in the agreement ID, and **_hashTypedDataV4** does not include it in the signed data either. Anyone can create the same agreement ID with a different expiry:

```
contractId = keccak256(
    abi.encode(templateId, salt, globalValues, parties, secretHash, finalizer)
);
```

An attacker can front-run the standalone creation with a much later expiry. The victim's transaction then reverts because the agreement ID already exists, but the signature exposed in its calldata still verifies for the attacker's agreement. Since standalone agreements have no finalizer, anyone can relay that signature through **signContractFor**. A remaining party can then sign after the victim's intended deadline and finalize the agreement.

One possible attack looks like this:

1. Alice broadcasts a transaction that creates and signs a two-party standalone agreement with Bob, expiring at time **T**.
2. Bob, or a cooperating searcher, front-runs it and creates the same agreement ID with an expiry later than T. The agreement can initially use an empty partyValues array.
3. Alice's transaction reverts with **ContractAlreadyExists**, but her signature is now public.
4. The attacker relays Alice's signature to the pre-created agreement using **signContractFor**.

5. After **T**, Bob signs. The attacker-controlled expiry is still valid, so the registry finalizes an agreement that should already have expired.

```
function createStandaloneContractAndSignFor(
    string memory title,
    string memory legalContractUri,
    string[] memory globalFields,
    string[] memory partyFields,
    uint256 salt,
    string[] memory globalValues,
    address[] memory parties,
    string[][] memory partyValues,
    uint256 expiry,
    address signer,
    bytes calldata signature
) public returns (bytes32 contractId) {
    // Derive template ID
    bytes32 templateId = keccak256(abi.encode(
        title,
        legalContractUri,
        globalFields,
        partyFields
    ));

    // Create the template if needed
    if (bytes(templates[templateId].legalContractUri).length == 0) {
        _createTemplate(
            templateId,
            title,
            legalContractUri,
            globalFields,
            partyFields
        );
    }

    contractId = createContract(
        templateId,
        salt,
        globalValues,
        parties,
        partyValues,
        "", // secretHash
        address(0), // fixed finalizer, see notice above
        expiry
    );
```

```
        signContractFor(
            signer,
            contractId,
            partyValues[0], // proposer
            signature,
            false, // proposer should explicitly add himself to the parties
            "" // secret
        );
    }
```

Remediation:

Include the absolute expiry, or a separate signer-approved deadline, in the EIP-712 signed data and reject the signature after that deadline. The expiry may also be included in the agreement ID as additional protection.

MTLX1-35 | Upgrading certificate printers freezes custody transfers of legacy certificates

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

metalex-jul-26/src/storage/LedgerEntryTokenStorage.sol:recordHolderChange#L755-L775

Description:

recordHolderChange tracks each address's certificate balance and the total number of unique holders. **LedgerEntryToken._update** calls it for every mint, burn, and transfer, including **transferFrom**, **safeTransferFrom**, and endorsed transfers. It therefore affects holders and custodians of certificates minted before their printer beacon is upgraded.

After an upgrade, the existing ERC-721 ownership should stay in sync with any newly added holder accounting. A legacy holder's counter should be initialized from their existing balance before a transfer decrements it, so the certificate can move normally and both holder counts remain correct.

However, the upgrade keeps the old ERC-721 ownership but does not migrate the new counters, which remain zero. On the first non-self transfer from a legacy holder, **recordHolderChange** executes:

```
uint256 fromBalance = s.holderTokenCount[from] - 1;
```

This evaluates to **0 - 1** and reverts. Since the call happens before the parent ERC-721 **_update**, ownership is never changed. Transfer approvals, endorsements, and transferability settings cannot work around the issue because all transfer paths use the same hook. A legacy primary deal can also accept the buyer's payment after the upgrade but fail when finalization tries to deliver the certificate, leaving the payment in escrow until the deal is voided and refunded or the implementation is fixed.

One failure scenario is:

1. Before the upgrade, an issuer calls **proposeDeal**, which mints a certificate to the DealManager.
2. The issuer upgrades the certificate-printer beacon to the new implementation, but the DealManager's holder counter remains zero.
3. The buyer calls **signDealAndPay**, moving their payment into the deal's **PAID** escrow.
4. **finalizeDeal** tries to transfer the certificate to the buyer. The call reaches **_update** and then

recordHolderChange(DealManager, buyer).

5. The zero counter underflows, finalization reverts, and the certificate and payment remain in their respective escrows.

```
function recordHolderChange(address from, address to) internal {
    CyberCertStorage storage s = cyberCertStorage();

    if (from == to) return;

    if (from != address(0)) {
        uint256 fromBalance = s.holderTokenCount[from] - 1;
        s.holderTokenCount[from] = fromBalance;
        if (fromBalance == 0) {
            s.uniqueHolderCount--;
        }
    }

    if (to != address(0)) {
        uint256 toBalance = s.holderTokenCount[to];
        if (toBalance == 0) {
            s.uniqueHolderCount++;
        }
        s.holderTokenCount[to] = toBalance + 1;
    }
}
```

Remediation:

Add a batched, idempotent migration that initializes each legacy holder's token count and the global unique-holder count before the new accounting is enabled. Until migration is complete, gate this accounting or lazily initialize a sender from the existing ERC-721 balance before decrementing it.

MTLX1-38 | Upgrading a live CyberScrip can block complete redemption

Acknowledged

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

metalex-jul-26/src/CyberScrip.sol:_update#L60-L105

Description:

CyberScrip._update handles regular mints, transfers, and burns. It also tracks how many addresses hold scrip. When a user calls **IssuanceManager.convertScripToCert**, the flow ends in **CyberScrip.burnFrom**, which reaches **_update** and lowers the holder count if the user's full balance is burned.

For a newly deployed scrip, each first mint increases **holderCount**, and a holder's full transfer or burn decreases it. This keeps the counter in sync with the real number of holders and lets all scrip eventually be redeemed.

However, the v4 implementation adds **holderCount** to the old v3 storage layout without migrating existing holders. A live v3 proxy therefore starts v4 with outstanding balances but **holderCount == 0**. When a legacy holder transfers or burns their full balance, **_update** tries to subtract one from zero and reverts:

```
uint256 holderDelta = s.holderCount;
bool decrementHolder = from != address(0) && fromBalanceBefore == amount;

if (decrementHolder) {
    holderDelta -= 1;
}
```

This also breaks a full transfer to a new address because the subtraction happens before the matching increment. The privileged **forceTransfer** and **forceBurn** functions are not a way out, as they perform the same checked subtraction in **_updateHolderCount**. Partial transfers or burns may move the point of failure, but they cannot fix the missing historical count. As a result, at least some scrip remains outstanding and operations that require totalSupply == 0, such as changing the scrip ratio, stay blocked until another upgrade fixes the state.

One failure path is:

1. Alice receives 100 scrip while the proxy still uses v3.

2. The issuer upgrades the existing scrip beacon to v4. Alice keeps her balance, but **holderCount** remains zero.
3. Alice calls **convertScripToCert** for all 100 scrip, which calls **burnFrom(Alice, 100)**.
4. **_update** identifies Alice as an exiting holder and evaluates **0 - 1**, reverting the full conversion.

```
    function _update(address from, address to, uint256 amount) internal virtual override {
        CyberScripStorage.StorageData storage s = CyberScripStorage.getStorageData();
        uint256 fromBalanceBefore = 0;
        uint256 toBalanceBefore = 0;

        if (from != address(0)) {
            fromBalanceBefore = balanceOf(from);
        }
        if (to != address(0)) {
            toBalanceBefore = balanceOf(to);
        }

        // Enforce freeze checks for normal transfers (not mint/burn)
        if (to != address(0)) {
            if (s.canFreeze) {
                if (s.frozen[from]) revert AccountFrozen(from);
                if (s.frozen[to]) revert AccountFrozen(to);
            }
            // Enforce transfer restriction hooks
            uint256 length = s.transferRestrictionHooks.length;
            for (uint256 i = 0; i < length; i++) {
                (bool allowed, string memory reason) = s.transferRestrictionHooks[i].checkTransferRestriction(from,
to, amount, "");
                if (!allowed) revert RestrictedTransfer(reason);
            }
        }

        if (amount > 0 && from != to) {
            uint256 holderDelta = s.holderCount;
            bool decrementHolder = from != address(0) && fromBalanceBefore == amount;
            bool incrementHolder = to != address(0) && toBalanceBefore == 0;

            if (decrementHolder) {
                holderDelta -= 1;
            }
            if (incrementHolder) {
                holderDelta += 1;
            }
```

```
            if (s.maxHolderCount > 0 && holderDelta > s.maxHolderCount) {
                revert HolderLimitExceeded(s.maxHolderCount);
            }
        }

        super._update(from, to, amount);

        _updateHolderCount(from, to, amount, fromBalanceBefore, toBalanceBefore);
    }
```

Remediation:

Migrate the exact holder count before enabling the new accounting logic, and guard it with an explicit initialization flag. Add an upgrade test that creates balances under v3, upgrades the same proxy, and then fully transfers and redeems them until **totalSupply** reaches zero.

MTLX1-40 | Stale endorsements allow former owners to reclaim legal title

Fixed

Severity:	Probability:	Impact:
Medium	Unlikely	Medium

Path:

metalex-jul-26/src/storage/LedgerEntryTokenStorage.sol:processTransfer#L234-L286

Description:

processTransfer runs during ERC-721 transfers and decides whether delivery should also update the certificate's legal owner. If the sender is only the custodian, legal title is assigned to the recipient when the latest endorsement names that recipient.

An endorsement should only remain actionable while it is backed by the legal owner who authorized it. Likewise, when the registrar uses **assignCert** to assign legal title to someone else without moving the ERC-721, that newer assignment should not be reversible by the former owner.

However, **assignCert** updates the legal owner through **recordAssign** but does not clear or invalidate existing endorsements. Primary issuance normally records an endorsement naming the original investor, so this stale state exists without any special setup. If the original investor still holds the ERC-721 after a registrar assignment, they can self-transfer it. The custodian branch then treats the old endorsement as valid and changes legal title back to them:

```
else if (endorsementCount > 0 &&
    s.endorsements[tokenId][endorsementCount - 1].endorsee == to
) {
    Endorsement memory endorsement = s.endorsements[tokenId][endorsementCount - 1];
    _setLegalOwner(s, tokenId, endorsement.endorsee, endorsement.endorseeName);
}
```

One possible flow is:

1. Alice receives a certificate through primary issuance. She holds both the ERC-721 and legal title, and the latest endorsement names Alice.
2. The registrar calls **assignCert** to assign legal title to Carol. Alice still holds the ERC-721, and the endorsement to Alice remains active.
3. Alice calls **transferFrom(Alice, Alice, tokenId)**, which is a valid ERC-721 self-transfer.
4. **processTransfer** sees that Alice is not the current legal owner but that the transfer destination matches the stale endorsee, so it assigns legal title back to Alice.

```
    function processTransfer(address from, address to, uint256 tokenId) external {
        CyberCertStorage storage s = cyberCertStorage();

        // Check built-in transferability flag and per-token override
        if (!s.transferable && !s.tokenTransferable[tokenId]) {
            ICyberCorp corp = ICyberCorp(IIssuanceManager(s.issuanceManager).CORP());
            if (from != corp.dealManager() && from != corp.roundManager()) revert
ILedgerEntryToken.TokenNotTransferable();
        }

        // Check global hook if it exists
        if (address(s.globalRestrictionHook) != address(0)) {
            (bool allowed, string memory reason) = s.globalRestrictionHook.checkTransferRestriction(
                from, to, tokenId, ""
            );
            if (!allowed) revert ILedgerEntryToken.TransferRestricted(reason);
        }

        ITransferRestrictionHook typeHook =
LedgerEntryTokenStorage.cyberCertStorage().restrictionHooksById[tokenId];

        if (address(typeHook) != address(0)) {
            (bool allowed, string memory reason) = typeHook.checkTransferRestriction(
                from, to, tokenId, ""
            );
            if (!allowed) revert ILedgerEntryToken.TransferRestricted(reason);
        }

        address ownerAddress = s.owners[tokenId].ownerAddress;
        uint256 endorsementCount = s.endorsements[tokenId].length;
        //check endorsement and update owners
        if (from == ownerAddress) {
            if (!s.endorsementRequired) {
                emit ILedgerEntryToken.CertificateAssigned(tokenId, to, "",
IIssuanceManager(s.issuanceManager).companyName());
                _setLegalOwner(s, tokenId, to, "");
            }
            else if (endorsementCount > 0) {
                Endorsement memory endorsement = s.endorsements[tokenId][endorsementCount - 1];
                if (endorsement.endorsee == to) {
                    // Endorsement exists; ownership will be updated
                    emit ILedgerEntryToken.CertificateAssigned(tokenId, to, endorsement.endorseeName,
IIssuanceManager(s.issuanceManager).companyName());
                    _setLegalOwner(s, tokenId, endorsement.endorsee, endorsement.endorseeName);
```

```
            }
        }
        // NOTE: we don't revert in this block: Owner is able to transfer to another address without an endorsement, but it does not update the owner
    }
    // Token is not being transferred from the current owner (e.g. held by a custodian). Delivery to the party
    // named in the latest endorsement promotes legal title (DvP settlement); a move back to the legal owner or
    // on to any other party is a possession-only custody move that leaves legal ownership untouched.
    else if (endorsementCount > 0 && s.endorsements[tokenId][endorsementCount - 1].endorsee == to) {
        Endorsement memory endorsement = s.endorsements[tokenId][endorsementCount - 1];
        emit ILedgerEntryToken.CertificateAssigned(tokenId, to, endorsement.endorseeName, IIssuanceManager(s.issuanceManager).companyName());
        _setLegalOwner(s, tokenId, endorsement.endorsee, endorsement.endorseeName);
    }
}
```

Remediation:

Bind each endorsement to a protocol-controlled legal-ownership epoch and only accept it during that epoch. Increment the epoch, or otherwise invalidate the current actionable endorsement, whenever legal title genuinely changes.

MTLX1-31 | Mint callback allows recertification approval reuse

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

IssuanceManagerStorage.sol

Description:

convertScripToCert clears the recertification approval only after the certificate is minted. However, **safeMintAndAssign** calls **_safeMint** before recording the certificate's legal ownership. This triggers **onERC721Received** while the approval is still active and the new certificate is not yet visible in the legal-owner enumeration.

An attacker can therefore re-enter **convertScripToCert** from **onERC721Received** or any other function. The second call sees no active lot and reuses the same approval, allowing one officer-signed approval to mint multiple certificates.

Impact

This does not create free value, since each certificate still requires the corresponding amount of scrip. However, an approval intended for a single registration can be reused to create multiple certificates carrying the same officer signature and approved metadata.

Remediation:

Record the certificate state before triggering **onERC721Received**:

```
LedgerEntryTokenStorage.recordMintAndAssign(tokenId, owner, details, ownerName);
_safeMint(to, tokenId);


function safeMintAndAssign(
    address to, // custodian
    address owner, // legal owner
    uint256 tokenId,
    CertificateDetails memory details,
    string memory ownerName
) external onlyIssuanceManager returns (uint256) {
--  _safeMint(to, tokenId);
    LedgerEntryTokenStorage.recordMintAndAssign(tokenId, owner, details, ownerName);
++  _safeMint(to, tokenId);
    return tokenId;
}
```

MTLX1-30 | Permissionless agreement creation can block secondary trades

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

When a secondary offer is accepted, the **DealManager** derives the settlement salt from the public offer salt and the number of prior settlements. The remaining identifier inputs are also public or reconstructible.

```
function _acceptOffer(...) internal returns (...) {
    ...
    bytes32 settlementSalt = keccak256(abi.encodePacked(offer.salt, offer.settlementAgreementIds.length));
    address[] memory settlementParties = new address[](2);
    settlementParties[0] = offer.offeror;
    settlementParties[1] = acceptor;
    string[][] memory settlementPartyValues = new string[][](2);
    settlementPartyValues[0] = offer.offerorPartyValues;
    settlementPartyValues[1] = params.acceptorPartyValues;
    settlementAgreementId = ICyberAgreementRegistry(registry).createContract(
        offer.templateId,
        uint256(settlementSalt),
        offer.globalValues,
        settlementParties,
        settlementPartyValues,
        bytes32(0),
        address(this),
        settlementExpiry
    );
    ...
}
```

createContract is publicly callable and does not verify that the caller is the supplied finalizer or an authorized creator. The contract ID also excludes the caller, party values, and expiry.

```
function createContract(...) public returns (bytes32 contractId) {
    ...
    contractId = keccak256(
        abi.encode(templateId, salt, globalValues, parties, secretHash, finalizer)
```

```
        );
        if (agreements[contractId].parties.length > 0) {
            revert ContractAlreadyExists();
        }
        ...
    }
```

An unrelated account can therefore submit the same identifier inputs, specify the real **DealManager** as the finalizer, and occupy the expected agreement ID before the legitimate acceptance. The subsequent **acceptOffer** call reverts with **ContractAlreadyExists**.

Remediation:

- Require agreements with a nonzero finalizer to be created by that finalizer or an explicitly authorized creator.
- Namespace agreement IDs using the authenticated creator and acceptance-specific entropy that unrelated callers cannot reserve.

MTLX1-29 | Country policy updates do not invalidate cached zkPassport eligibility

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

src/libs/conditions/NonUSNationalityCondition.sol#L182

Description:

NonUSNationalityCondition checks a user's nationality against the current exclusion list only when a proof is submitted. The resulting cache stores an expiry timestamp, but no policy version identifying the exclusion list under which the proof was accepted.

```
    function updateExcludedCountries(string[] calldata _excludedCountries) external onlyAdmin {
        excludedCountries = _excludedCountries;
        emit ExcludedCountriesUpdated(_excludedCountries);
    }

    function submitProof(
        ProofVerificationParams calldata params,
        bool isIDCard
    ) external {
        ...
        if(!helper.isNationalityOut(excludedCountries, params.committedInputs)) revert
USAOrSanctionedCountriesNotAllowed();
        ...
        uint256 expiresAt = proofTimestamp + validityPeriod;
        ...
        proofExpiry[msg.sender] = expiresAt;
        uniqueIdentifierExpiry[uniqueIdentifier] = expiresAt;
        emit ProofSubmitted(msg.sender, expiresAt);
    }
```

At settlement time, **checkCondition** considers only a founder override or the cached expiry. It does not verify that the proof was accepted under the current country policy.

```
    function checkCondition(
        address _contract,
```

```
        bytes4,
        bytes memory data
    ) public view override returns (bool) {
        ...
        if (founderOverrides[_contract][counterparty]) return true;
        return proofExpiry[counterparty] >= block.timestamp;
    }
```

Consequently, a user whose country is added to the exclusion list can continue passing the condition for new gated deals until the cached proof expires. A fresh proof is evaluated against the updated policy, while an existing unexpired cache remains valid.

Remediation:

- Store the current policy version with each accepted proof and require it to match during condition checks.
- Increment the policy version when the exclusion list or validity policy changes.
- If existing proofs should remain valid after a policy update, implement explicit prospective and immediate update modes with defined cutoff behavior.

MTLX1-28 | Certificate unit movements do not preserve partly paid balances

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

src/storage/IssuanceManagerStorage.sol#L895-L918
src/storage/IssuanceManagerStorage.sol#L1108-L1111

Description:

ShareExtension stores the paid amount and total consideration as certificate-level values. Their difference represents the outstanding subscription balance.

```
struct CertificateData {
    bool isPartlyPaid;
    uint256 amountPaid;
    uint256 totalConsideration;
    string sourceAuthorityURI;
    ShareRepresentationType representationType;
    bool holdingPeriodTackingApplied;
}
```

During a partial secondary transfer, the seller's unit balance is reduced, but the complete **extensionData** is copied unchanged to the buyer's certificate. Both certificates therefore retain the original payment data, duplicating the recorded paid amount, total consideration, and outstanding balance.

```
    function executeSecondaryTransfer(bytes calldata dealMetadata)
        external
        returns (uint256 buyerTokenId)
    {
        ...
        sellerDetails.unitsRepresented -= units;
        cert.updateCertificateDetails(tokenId, sellerDetails);
        ...
        CertificateDetails memory buyerDetails = CertificateDetails({
            signingOfficerName: sellerDetails.signingOfficerName,
            signingOfficerTitle: sellerDetails.signingOfficerTitle,
```

```
            investmentAmountUSD: 0,
            issuerUSDValuationAtTimeOfInvestment: 0,
            unitsRepresented: units,
            legalDetails: sellerDetails.legalDetails,
            extensionData: sellerDetails.extensionData
        });
        ...
    }
```

The payment data is also not preserved when units move through **CyberScrip.Scripification** removes units from the source certificate without updating its extension data, while conversion adds the units to an existing certificate without transferring or recalculating that data. A holder can therefore merge units from a partly paid certificate into a fully paid certificate while the outstanding balance remains attached to the empty source certificate.

```
    function executeScripifyCert(...) external
    {
        ...
        details.unitsRepresented = details.unitsRepresented - amount;
        certificate.updateCertificateDetails(id, details);
        ICyberScrip(scripifiedCert).mint(toSend, scripAmount);
        (uint256 newTotalAssetsWad, uint256 newTotalNominalShares) = getCertScripUnitVault(
            certAddress
        );
        emit ScripifiedCert(
            certAddress,
            id,
            scripifiedCert,
            amount,
            details.unitsRepresented,
            getScripPoolSharesById(certAddress, id),
            newTotalAssetsWad,
            newTotalNominalShares
        );
    }

    function executeConvertScripToCert(...) external
    {
        ...
        if (selection.foundActive) {
            CertificateDetails memory activeDetails = certificate
                .getActiveCertificateDetails(selection.activeTokenId);
```

```
        activeDetails.unitsRepresented =
            activeDetails.unitsRepresented +
            units;
        certificate.updateCertificateDetails(
            selection.activeTokenId,
            activeDetails
        );
        ...
    }
```

Remediation:

- Allocate **amountPaid** and **totalConsideration** when certificate units are split, including deterministic remainder handling.
- Reject partial transfers when the attached extension data cannot be divided safely.
- Prevent partly paid units from entering fungible **CyberScrip**, or preserve their payment provenance in separate tranches through recertification.

MTLX1-27 | CyberScrip applies transfer hooks to mints

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

CyberScrip applies transfer restriction hooks during mints. Since mints use **from = address(0)**, the repo's **WhitelistTransferHook** rejects every mint because **address(0)** is not whitelisted.

```
if (to != address(0)) {
    // transfer hooks
}
```

The condition should exclude both mints and burns:

```
if (from != address(0) && to != address(0)) {
    // transfer hooks
}
```

Impact

When **WhitelistTransferHook** is enabled, scrip issuance always reverts. Existing scrip and transfers are unaffected.

Remediation:

Apply transfer hooks only when both **from** and **to** are non-zero, while keeping the existing freeze check for mints.

MTLX1-26 | Round authorization can be reused through a factory with weaker metadata validation

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

The authority officer's escrowed signature is verified against the **RoundManager** address and a limited set of round parameters. It does not cover the factory used to deploy the round or related metadata such as companyPayable, roundConditions, certData, legalDetails, extensionData, roundPartyValues, and the round policy flags.

```
    bytes32 constant ESCROWEDSIGNATUREDATA_TYPEHASH = keccak256(
        "EscrowedSignatureData(bytes32 roundId,uint8 seriesType,uint256 raiseCap,uint256 minTicket,uint256
maxTicket,uint8 roundType,uint256 startTime,uint256 endTime,bytes32 templateId,address
paymentToken,uint256 pricePerUnit,uint256 valuation,address companyAddress)"
    );
```

PumpCorpFactory.deployCyberCorpAndCreateRoundFor protects the additional deployment fields with a separate officer-signed metadata payload. In contrast, **CyberCorpFactory.deployCyberCorpAndCreateRound** is public, accepts the officer and deployment metadata from the caller, and creates the round without verifying the supplemental signature.

```
    function deployCyberCorpAndCreateRound(
        uint256 salt,
        SecuritySeries seriesType,
        string memory companyName,
        string memory companyType,
        string memory companyJurisdiction,
        string memory companyContactDetails,
        string memory defaultDisputeResolution,
        address _companyPayable,
        CompanyOfficer memory _officer,
        string[] memory legalDetails,
        bytes[] memory extensionData,
        RM_CyberCertData[] memory certData,
        bytes32 templateId,
        address paymentToken,
        uint256 pricePerUnit,
```

```
    uint256 valuation,
    string[] memory roundPartyValues,
    bytes memory escrowedSignature,
    RoundType roundType,
    address[] memory conditions,
    uint256 raiseCap,
    uint256 minTicket,
    uint256 maxTicket,
    uint256 startTime,
    uint256 endTime,
    bool publicRound,
    bool allowTimedOffers,
    bool restrictEndTimeReduction
)
    external
    returns (
        address cyberCorpAddress,
        address authAddress,
        address issuanceManagerAddress,
        address dealManagerAddress,
        address roundManagerAddress,
        bytes32 roundId
    )
{
    bytes32 corpSalt = keccak256(abi.encodePacked(salt));

    (
        cyberCorpAddress,
        authAddress,
        issuanceManagerAddress,
        dealManagerAddress,
        roundManagerAddress
    ) = deployCyberCorp(
        corpSalt,
        companyName,
        companyType,
        companyJurisdiction,
        companyContactDetails,
        defaultDisputeResolution,
        _companyPayable,
        _officer
    );
```

```
        // Deploy RoundManager via its factory
        bytes32 rmSalt = keccak256(abi.encodePacked("round", salt));


        // Create round with provided round type using RoundLib
        {
            Round memory draft = RoundLib
                .draft()
                .setTickets(
                    seriesType,
                    roundType,
                    publicRound,
                    allowTimedOffers,
                    restrictEndTimeReduction,
                    raiseCap,
                    minTicket,
                    maxTicket,
                    paymentToken,
                    pricePerUnit,
                    valuation,
                    startTime,
                    endTime
                )
                .setAgreement(
                    templateId,
                    _officer.eoa,
                    _officer.name,
                    _officer.title,
                    legalDetails,
                    roundPartyValues,
                    extensionData,
                    conditions,
                    escrowedSignature
                );
            roundId = IRoundManagerInterface(roundManagerAddress).createRound(
                draft,
                certData
            );
        }
    }
```

When both top-level factories use the same sub-factories and implementation references, the same salt produces the same **CyberCorp** and **RoundManager** addresses. A relayer holding a

a valid **PumpCorp** signature package can therefore submit the escrowed signature through **CyberCorpFactory** while replacing metadata covered only by the **PumpCorp** supplemental signature.

The relayer cannot alter the signed round parameters or obtain the officer's role. However, the resulting round may contain a different payout address, condition set, or certificate configuration. If the round is funded before the officer corrects the mutable settings, settlement uses the substituted **companyPayable**.

Remediation:

- Require every public round-deployment path to verify officer authorization over the complete deployment metadata.
- Bind the authorized top-level factory or deployment purpose and the corporation salt to the signed payload.
- Include the payout address, officer metadata, agreement values, certificate configuration, conditions, and round policy flags in the validated commitment.
- Prevent factories with different authorization requirements from recreating the same signed deployment identity.

MTLX1-25 | Voided certificates can be used in secondary trades

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

src/storage/SecondaryTradeStorage.sol#L319

Description:

The secondary-trade flow verifies the seller's registered ownership and available units but does not verify that the source certificate remains active.

When accepting a BUY offer, the seller selects the certificate used for settlement. The protocol checks that the seller is its registered owner and reserves the requested units, without checking **isVoided**.

```
function _acceptOffer(...) internal returns (bytes32 settlementAgreementId) {
    ...

    if (offer.side == OfferSide.SELL) {
        ...
    } else {
        certPrinter = offer.certPrinter;
        tokenId = params.sellerTokenId;
        buyer = offer.offeror;
        endorsementSig = params.openEndorsementSig;
        if (ILedgerEntryToken(certPrinter).legalOwnerOf(tokenId) != acceptor)
            revert ISecondaryTradeStorage.NotCertOwner();
        ILedgerEntryToken(certPrinter).increaseUnitsReserved(tokenId, params.units);
    }
```

Voiding a certificate changes its security status but preserves its registered owner and certificate details. A voided certificate can therefore pass the ownership and unit checks during acceptance and finalization.

During settlement, **executeSecondaryTransfer** reads the retained details, deducts the transferred units from the voided certificate, and mints a new assigned certificate for the buyer.

```
function executeSecondaryTransfer(bytes calldata dealMetadata)
    external
    returns (uint256 buyerTokenId)
{
    ...
    CertificateDetails memory sellerDetails = cert.getActiveCertificateDetails(tokenId);
    if (units > sellerDetails.unitsRepresented) revert AmountExceedsAvailableUnits();
    sellerDetails.unitsRepresented -= units;
    cert.updateCertificateDetails(tokenId, sellerDetails);
    bool sellerVoided = sellerDetails.unitsRepresented == 0;
    if (sellerVoided) {
        cert.voidCert(tokenId);
    }

    ...

    (, buyerTokenId) = _mintAssignedCert(certPrinter, custodian, buyer, buyerDetails, buyerName);

    ...
}
```

As a result, a holder can use an issuer-voided certificate to fill a funded BUY offer, receive the buyer's payment, and cause the transferred units to appear in a new active certificate.

Remediation:

- Require the source certificate to have an active status when posting a SELL offer and accepting a BUY offer.
- Recheck the certificate status immediately before payment distribution and secondary transfer execution.
- Enforce the same active-status requirement within the issuance manager's secondary-transfer path.

MTLX1-24 | Issuer authorization does not bind investor-supplied agreement terms

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

When a round is created, the authority officer provides an EIP-712 signature authorizing its economic and operational parameters. The signed structure covers the round identifier, ticket limits, round period, payment token, price, valuation, and company address. It does not include a hash of the agreement's **globalValues** or the complete agreement body.

```
bytes32 constant ESCROWEDSIGNATUREDATA_TYPEHASH = keccak256(
    "EscrowedSignatureData(bytes32 roundId,uint8 seriesType,uint256 raiseCap,uint256 minTicket,uint256
maxTicket,uint8 roundType,uint256 startTime,uint256 endTime,bytes32 templateId,address
paymentToken,uint256 pricePerUnit,uint256 valuation,address companyAddress)"
  );
```

The agreement values are instead supplied by the investor through **submitEOI**. Depending on the selected template, these values may represent terms such as the purchase amount, valuation cap, expiration, governing jurisdiction, dispute resolution, or warrant terms. The **RoundManager** forwards the values directly to the registry when creating the agreement.

For FCFS rounds, it then calls **signContractWithEscrow** using the officer's existing round signature.

```
        agreementId = ICyberAgreementRegistry(ls.DEAL_REGISTRY)
            .createContract(
                round.templateId,
                salt,
                globalValues,
                parties,
                partyValuesArray,
                secretHash,
                address(this),
                expiry
            );

        Token[] memory corpAssets = new Token[](0);
        Token[] memory buyerAssets = new Token[](1);
        buyerAssets[0] = Token(
```

```
            TokenType.ERC20,
            round.paymentToken,
            0,
            eoi.maxAmount,
            true // Will be used as fee token
        );

        LexScrowStorage.createEscrow(agreementId, counterParty, corpAssets, buyerAssets, expiry);

        if (round.roundType == RoundType.FCFS) {
            ICyberAgreementRegistry(ls.DEAL_REGISTRY)
                .signContractWithEscrow(
                    round.authorityOfficer,
                    agreementId,
                    round.roundPartyValues,
                    round.escrowedSignature,
                    false,
                    ""
                );
        }
```

Although **signContractWithEscrow** receives a signature parameter, it does not verify that signature against the agreement's template, global values, or party values. After confirming that the caller is the configured finalizer and that the supplied party values have the expected length, the function records the officer as having signed the agreement.

```
    function signContractWithEscrow(
        address escrowSigner,
        bytes32 contractId,
        string[] memory partyValues,
        bytes calldata signature,
        bool fillUnallocated,
        string memory secret
    ) onlyDefinedFinalizer(contractId) external {
        ...
        uint256 timestamp = block.timestamp;

        agreementData.partyValues[escrowSigner] = partyValues;
        agreementData.signedAt[escrowSigner] = timestamp;
        uint256 totalSignatures = ++agreementData.numSignatures;

        emit AgreementSigned(contractId, escrowSigner, timestamp);
    }
```

An investor can therefore provide agreement terms that were not included in the officer's authorization, while the resulting agreement records the officer as a signer. In an FCFS round, the same transaction can proceed through payment, allocation, agreement finalization, and certificate issuance. Founder-approved rounds use the same signature path when an allocation is approved.

Remediation:

- Bind a hash of the complete agreement body, including the relevant global and issuer party values, to the authority officer's signed authorization.
- Verify the officer's signature against the exact agreement data before recording the officer as a signer.
- Enforce the same agreement-binding validation in both FCFS and FounderApproved allocation paths.

MTLX1-22 | Buyer-selected DealManager custody permits transfers outside the secondary settlement flow

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

src/storage/SecondaryTradeStorage.sol#L442-L450

Description:

For administered secondary trades, the buyer supplies the **adminMultisig** address used as the certificate custodian. The value is accepted without preventing the **DealManager** from being selected.

```
    function _acceptOffer(AcceptOfferParams calldata params, address acceptor) internal returns (bytes32
settlementAgreementId) {
        ...
        string memory buyerName;
        HostingMode buyerHostingMode;
        address adminMultisig;
        if (offer.side == OfferSide.BUY) {
            buyerName = offer.buyerName;
            buyerHostingMode = offer.buyerHostingMode;
            adminMultisig = offer.adminMultisig;
        } else {
            buyerName = params.buyerName;
            buyerHostingMode = params.buyerHostingMode;
            adminMultisig = params.adminMultisig;
        }
        ...
```

The **IssuanceManager** mints the certificate to the selected custodian while recording the buyer as its holder of record.

```
    function executeSecondaryTransfer(bytes calldata dealMetadata)
        external
        returns (uint256 buyerTokenId)
    {
        ...
```

```
        bool buyerTokenIsMinted = true;
        address custodian = buyerHostingMode == HostingMode.ADMINISTERED ? adminMultisig : buyer;
        CertificateDetails memory buyerDetails = CertificateDetails({
            signingOfficerName: sellerDetails.signingOfficerName,
            signingOfficerTitle: sellerDetails.signingOfficerTitle,
            investmentAmountUSD: 0,
            issuerUSDValuationAtTimeOfInvestment: 0,
            unitsRepresented: units,
            legalDetails: sellerDetails.legalDetails,
            extensionData: sellerDetails.extensionData
        });
        (, buyerTokenId) = _mintAssignedCert(certPrinter, custodian, buyer, buyerDetails, buyerName);
        ...
```

A buyer can therefore select the **DealManager** as custodian, resulting in the **DealManager** holding the ERC-721 while the buyer remains the holder of record. The buyer can add an endorsement and invoke **endorseAndTransfer**, which uses an internal transfer without requiring authorization from the ERC-721 custodian.

```
    function addEndorsement(uint256 tokenId, Endorsement memory newEndorsement) public {
        if(msg.sender != LedgerEntryTokenStorage.cyberCertStorage().issuanceManager && msg.sender != legalOwnerOf(tokenId)) revert ILedgerEntryToken.InvalidEndorsement();
        LedgerEntryTokenStorage.recordEndorsement(tokenId, newEndorsement);
    }

    function endorseAndTransfer(uint256 tokenId, Endorsement memory newEndorsement, address from, address to) external {
        addEndorsement(tokenId, newEndorsement);
        _transfer(from, to, tokenId);
    }
```

The non-transferability check permits this operation based on the from address being the **DealManager**, rather than requiring the caller to be an authorized manager executing a settlement.

```
    function processTransfer(address from, address to, uint256 tokenId) external {
        CyberCertStorage storage s = cyberCertStorage();

        if (!s.transferable && !s.tokenTransferable[tokenId]) {
            ICyberCorp corp = ICyberCorp(IIssuanceManager(s.issuanceManager).CORP());
            if (from != corp.dealManager() && from != corp.roundManager()) revert ILedgerEntryToken.TokenNotTransferable();
```

```
    }
```

This allows the buyer to transfer its acquired certificate and update the on-chain holder-of-record entry without the custodian's approval or the **DealManager** agreement, secondary-condition, and fee logic.

Remediation:

- Bind the **DealManager** and **RoundManager** transfer exemptions to authorized manager callers or settlement operations rather than the token's custody address.
- Validate administered custodian addresses and prevent protocol manager contracts from being selected unless an explicit custody flow supports them.
- Require custodian authorization for **endorseAndTransfer**, or route holder-initiated transfers through an interface that enforces the custodian's transfer policy.

MTLX1-20 | Single admin can bypass the two-key kill switch

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

KillSwitchCondition requires two different admins to lower a kill switch. However, **rotateAdmin()** changes the admin address without invalidating the existing **lowerProposer**.

```
function rotateAdmin(address newAdmin) external onlyKillAdmin {
    ...
    if (msg.sender == metalexAdmin) {
        metalexAdmin = newAdmin;
    } else {
        legionAdmin = newAdmin;
    }
    // lowerProposer is not cleared
}
```

This allows one admin to bypass the two-key requirement:

1. Admin A: proposeLower()
2. Admin A: rotateAdmin(newAddress)
3. New address: confirmLower()

confirmLower() only checks that the caller is different from the stored proposer:

```
if (msg.sender == proposer) revert ProposerCannotConfirm();
killed = false;
```

After rotation, the new address is different from **lowerProposer**, so the check passes and the kill switch is lowered by the same admin alone. The same issue exists for settlement-level kills.

MTLX1-18 | Look-through tally can be corrupted if the badge is wired late

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

./src/LedgerEntryTokenStorage.sol

Description:

ICA §3(c)(1) limits a fund to 100 beneficial owners, not 100 addresses. To avoid looping over every holder on each trade, the contract keeps a cached **lookThroughHolderCount**.

This count depends on:

- **lookThroughBadge**: tells the contract how many beneficial owners a holder represents.
- **liveCounted**: prevents a token from being counted twice.
- **backfillLookThroughTally()**: counts existing tokens after an upgrade.

The problem is that the contract can count a token before the badge is configured. In that case, the holder is recorded with the default weight of 1 and marked **liveCounted**. Once marked, the backfill will skip the token even after the correct badge is configured.

Path A:

1. A beacon upgrade adds the tally to existing printers. The badge is initially unset.
2. Before the operator configures it, anyone can call the external, permissionless **backfillLookThroughTally()**.
3. Existing holders are recorded with weight 1 and marked **liveCounted**.
4. The operator configures the correct badge and runs the backfill again, but the tokens are skipped.

Result: Two legal owners representing 80 BOs can remain counted as only 2.

Path B

The same issue occurs if a new printer mints certificates before **setLookThroughBadge()**. Those holders are also recorded with weight 1 and cannot be fixed by the later backfill.

Impact: The fund's beneficial-owner count can be severely undercounted, allowing trades that exceed the intended holder cap. The U.S. holder count can also be overstated, causing valid trades to be rejected. No funds are directly stolen, but the on-chain compliance record becomes incorrect.

Remediation:

Do not allow minting or **backfillLookThroughTally()** to count holders while the badge is unset. If **address(0)** is a valid disabled state, use an explicit flag to distinguish it from an uninitialized badge.

MTLX1-21 | Auto-mint transfer failure blocks large investor allocations

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

RoundManagerStorage.sol, lexchexMinter.sol

Description:

When a large new investor is allocated, **allocate()** automatically requests a LeXcheX credential using mint parameters supplied by the investor.

The problem is that **requestMintFor()** is called from the RoundManager, making the RoundManager the **msg.sender** of the minter. The minter then attempts to transfer the credential fee from the RoundManager rather than from the investor.

```
if (!ILexCheX(getLexChex()).hasValidLexCheX(escrow.counterParty)) {
    if (IRoundManagerFactory(getUpgradeFactory()).isWhitelistedToken(round.paymentToken)) {

        if (usedAmount1e18 >= 200000 * 1e18 && eoi.naturalPerson) {
            (, tokenId) = ILexCheXMinter(getLexChexMinter()).requestMintFor(
                eoi.lexchexDetails.request,
                eoi.lexchexDetails.templateId,
                eoi.lexchexDetails.salt,
                eoi.lexchexDetails.globalValues,
                eoi.lexchexDetails.parties,
                eoi.lexchexDetails.partyValues,
                eoi.lexchexDetails.agreementSignature
            );
        }

        if (usedAmount1e18 >= 1000000 * 1e18 && !eoi.naturalPerson) {
            (, tokenId) = ILexCheXMinter(getLexChexMinter()).requestMintFor(
                eoi.lexchexDetails.request,
                eoi.lexchexDetails.templateId,
                eoi.lexchexDetails.salt,
                eoi.lexchexDetails.globalValues,
                eoi.lexchexDetails.parties,
                eoi.lexchexDetails.partyValues,
                eoi.lexchexDetails.agreementSignature
```

```
            );
        }
    }
}
```

Inside **requestMintFor()**, the payment is taken from **msg.sender**:

```
if (request.mintPrice > 0) {
    IERC20(request.paymentToken).safeTransferFrom(
        msg.sender,
        treasury,
        request.mintPrice
    );
}
```

Because **requestMintFor()** is called by **RoundManager**, the effective flow is:

```
Investor
  |
  | submitEOI()
  ▼
RoundManager
  |
  | requestMintFor()
  ▼
LeXcheXMinter
  |
  | safeTransferFrom(msg.sender ( round maanger ), ...)
  ▼
```

Thus, the investor is not the payer. The RoundManager is.

Since the **RoundManager** has no allowance for the minter, a non-zero **mintPrice** causes the transfer to revert with **ERC20InsufficientAllowance**.

Impact

This can prevent allocations for large new investors when the auto-mint branch is reached:

- Individual allocation ≥ $200k
- Entity allocation ≥ $1M
- Investor does not already have a valid LeXcheX
- Round payment token is whitelisted

For FCFS rounds, the failed mint can cause the investor's **submitEOI()** transaction to revert, preventing the large allocation.

Remediation:

Ensure the mint fee is deducted from that investor's allocation rather than the RoundManager's shared escrow balance.

MTLX1-2 | Burning a state credential makes a blocked buyer appear non-U.S.

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

src/libs/conditions/secondary/USStateOfResidenceCondition.sol#L80

Description:

LeXcheXBadge supports per-category burn authorization. When the category governing **ATTR_US_STATE** is configured with **OwnerOnly** or **Both**, the holder can burn the authoritative state credential. Burning removes both the token and its credential record.

```
    function burn(uint256 tokenId) public {
        address owner = _requireOwned(tokenId);
        BurnAuth auth = burnAuth(tokenId);

        if (auth == BurnAuth.Neither) revert LexChexBadge_TokenCannotBeBurned();
        if (auth == BurnAuth.OwnerOnly && msg.sender != owner) revert LexChexBadge_OnlyOwnerCanBurn();
        if (auth == BurnAuth.IssuerOnly && !_isIssuer(msg.sender)) revert LexChexBadge_OnlyIssuerCanBurn();
        if (auth == BurnAuth.Both && msg.sender != owner && !_isIssuer(msg.sender)) {
            revert LexChexBadge_OnlyOwnerCanBurn();
        }

        _burn(tokenId);
        LeXcheXBadgeStorage.deleteCredential(tokenId);
        emit CredentialBurned(owner, tokenId);
    }
```

_mostRecentValidWith distinguishes whether a valid authoritative credential was found, but **getUsState** discards this flag and returns **bytes2(0)** when none exists. The same value is also used for a legitimately attested non-U.S. holder.

```
    function _mostRecentValidWith(address owner, uint256 attributeMask) internal view returns (uint256
tokenId, bool found) {
        uint64 latest = 0;
        uint256 balance = balanceOf(owner);
        for (uint256 i = 0; i < balance; i++) {
```

```
            uint256 candidate = tokenOfOwnerByIndex(owner, i);
            if (!isValid(candidate)) continue;
            Credential storage cred = LeXcheXBadgeStorage.getCredential(candidate);
            // filter only categories governing ALL attributes selected by attributeMask
            if ((LeXcheXBadgeStorage.getCategory(cred.categoryId).governedAttributes & attributeMask) !=
attributeMask) continue;
            if (!found || cred.lastUpdated > latest || (cred.lastUpdated == latest && candidate > tokenId)) {
                latest = cred.lastUpdated;
                tokenId = candidate;
                found = true;
            }
        }
    }
```

USStateOfResidenceCondition interprets **bytes2(0)** as non-U.S. and immediately passes the buyer. Consequently, a buyer from a blocked state can burn its sole holder-burnable state credential and become indistinguishable from a non-U.S. buyer.

```
    function checkCondition(
        IDealManager dealManager,
        bytes4,
        bytes32 offerId,
        bytes32 agreementId
    ) external view override returns (bool) {
        Offer memory offer = dealManager.getOffer(offerId);
        (, address buyer,) = _resolveParties(dealManager, offer, agreementId);

        // No acquirer yet (posting context) — nothing to gate
        if (buyer == address(0)) return true;

        // Non-U.S. acceptors carry no usState attribute (enforced at badge mint): silent
        bytes2 state = badge.getUsState(buyer);
        if (state == bytes2(0)) return true;

        return !isStateBlocked(offer.spvAddress, state);
    }
```

Where eligibility is maintained through the independent **EligibilityCondition.cleared** mapping and other conditions require separate credential categories, removing the state credential does not invalidate those gates. Acceptance and finalization evaluate the same zero-state value, allowing the blocked-state buyer to complete the settlement.

The same absence/default ambiguity exists in **getBeneficialOwnerCount**, **getInvestorJurisdiction**, and **getRegulatoryJurisdiction**, although the security impact depends on how each value is consumed.

Remediation:

- Expose attribute presence separately from its value and fail closed when no valid authoritative credential exists.
- Determine non-U.S. status from a positive jurisdiction or **NON_US_PERSON** attestation instead of inferring it solely from usState == 0.
- Require categories governing **ATTR_US_STATE** to enforce a nonzero state for U.S. credentials.
- Preserve presence information for the other default-valued getters and apply consumer-specific fail-closed handling.

MTLX1-4 | Buyer revocation does not invalidate an existing settlement authorization

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Description:

When the company has signed a pending deal, a buyer's **revokeDeal** call only records the buyer in **voidRequestedBy**. The agreement is not marked as voided because the buyer is not **parties[0]** and the other party has not requested revocation.

```
function voidContractFor(
    bytes32 contractId,
    address party,
    bytes calldata signature
) public {
    ...

    agreementData.voidRequestedBy.push(party);
    emit VoidRequested(contractId, party);

    if (agreementData.expiry < block.timestamp) {
        agreementData.voided = true;
    } else if (
        agreementData.voidRequestedBy.length ==
        agreementData.parties.length &&
        agreementData.voidRequestedBy.length > 0
    ) {
        agreementData.voided = true;
    } else if (
        agreementData.parties[0] == party &&
        agreementData.numSignatures == 1
    ) {
        agreementData.voided = true;
    }

    if (agreementData.voided)
        emit ContractVoided(
            contractId,
            agreementData.voidRequestedBy,
            block.timestamp
        );
}
```

The relayed settlement path only checks whether the agreement is fully voided. It does not reject a signer recorded in **voidRequestedBy**. A caller holding the buyer's existing signature can therefore submit it through **signAndFinalizeDeal**. If the buyer retains sufficient balance and allowance and the configured conditions pass, the function collects payment and finalizes the deal despite the earlier revocation request.

```
    function signAndFinalizeDeal(
        address signer,
        bytes32 agreementId,
        string[] memory partyValues,
        bytes memory signature,
        bool _fillUnallocated,
        string memory name,
        string memory secret
    ) public {
        if (!LexScrowStorage.hasPrimaryEscrow(agreementId)) revert LexScrowStorage.DealDoesNotExist();
        address registry = LexScrowStorage.getDealRegistry();
        if(ICyberAgreementRegistry(registry).isVoided(agreementId)) revert LexScrowStorage.DealVoided();
        if(ICyberAgreementRegistry(registry).isFinalized(agreementId)) revert
LexScrowStorage.DealAlreadyFinalized();
        if(LexScrowStorage.getEscrow(agreementId).status != EscrowStatus.PENDING) revert
IDealManagerStorage.DealNotPending();

        string[] storage counterPartyCheck = getCounterPartyValues(agreementId);
        if(counterPartyCheck.length > 0) {
            if (keccak256(abi.encode(counterPartyCheck)) != keccak256(abi.encode(partyValues))) revert
IDealManagerStorage.CounterPartyValueMismatch();
        } else {
            setCounterPartyValues(agreementId, partyValues);
        }

        if (!ICyberAgreementRegistry(registry).hasSigned(agreementId, signer)) {
            // Not signed in registry yet; enforce local consistency and then sign
            ICyberAgreementRegistry(registry).signContractFor(signer, agreementId, partyValues, signature,
_fillUnallocated, secret);
        } else {
            // Already signed in registry; fetch values recorded in the registry and ensure consistency
            string[] memory registryValues = ICyberAgreementRegistry(registry).getSignerValues(agreementId,
signer);
            if (keccak256(abi.encode(registryValues)) != keccak256(abi.encode(partyValues))) revert
IDealManagerStorage.CounterPartyValueMismatch();
        }

        LexScrowStorage.updateEscrow(agreementId, signer, name);
```

```
        if(!LexScrowStorage.conditionCheck(agreementId)) revert
ILexScrowStorage.AgreementConditionsNotMet();
        LexScrowStorage.handleCounterPartyPayment(agreementId);
        finalizeDeal(agreementId);
    }
```

Remediation:

- Invalidate outstanding signer authorizations when **revokeDeal** succeeds, using a per-deal and per-signer nonce or equivalent revocation state.
- Reject signers recorded in **voidRequestedBy** before signature processing, payment collection, and finalization.
- Apply the revocation check consistently to both **signDealAndPay** and **signAndFinalizeDeal**.

MTLX1-7 | voidContractFor incorrectly voids zero-expiry agreements

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Path:

./src/CyberAgreementRegistry.sol

Description:

The registry treats **expiry == 0** as a valid agreement with no expiry.

```
function signContractFor(
    address signer,
    bytes32 contractId,
    string[] memory partyValues,
    bytes calldata signature,
    bool fillUnallocated, // to fill a 0 address or not
    string memory secret
) public {
<SNIP>
    if (agreementData.expiry > 0 && agreementData.expiry < block.timestamp)
```

However, **voidContractFor** compares the value directly against **block.timestamp**:

```
function voidContractFor(
    bytes32 contractId,
    address party,
    bytes calldata signature
) public {
    <SNIP>

    if (agreementData.expiry < block.timestamp) {
        agreementData.voided = true;
    }
```

Since **0 < block.timestamp** is always true, every zero-expiry agreement is incorrectly treated as expired and immediately voided.

Because the void logic uses an **if / else if** chain, execution stops at the expired branch and the normal authorization checks (unanimous consent or proposer withdrawal) are never evaluated. As a result, any party can permanently void a zero-expiry agreement during the signing phase, even though neither of the intended void conditions has been satisfied.

Remediation:

Consider changing the check to:

```
-- if (agreementData.expiry < block.timestamp) {
++ if (agreementData.expiry > 0 && agreementData.expiry < block.timestamp) {
```

MTLX1-12 | Zero adminMultisig Can Lock Secondary Trade Settlements

Fixed

Severity:	Probability:	Impact:
Low	Unlikely	Low

Description:

When a buyer selects **HostingMode.ADMINISTERED**, **adminMultisig** is used as the recipient of the newly minted certificate. However, the protocol does not validate that **adminMultisig** is non-zero.

A buyer can therefore accept an offer with **adminMultisig = address(0)**. The acceptance succeeds, but finalization later reverts because the certificate cannot be minted to the zero address.

By this point, the seller's units and buyer's payment are already locked in the settlement. The settlement cannot be repaired or finalized and remains stuck until its expiry (7 days by default).

After expiry, the buyer is fully refunded and the seller's units are released, allowing the same attack to be repeated against the offer.

Impact

An attacker can repeatedly prevent a seller's offer from being filled by accepting it with an invalid **adminMultisig**.

The attacker does not lose principal because the escrowed payment is fully refunded after expiry. The main cost is temporarily locking the required settlement amount.

This can also occur accidentally if a buyer selects administered custody without providing a multisig address.

Remediation:

Validate the custody parameters before accepting or posting the trade:

```
if (
    buyerHostingMode == HostingMode.ADMINISTERED &&
    adminMultisig == address(0)
) {
    revert InvalidAdminMultisig();
}
```

Apply this validation in both **_postOffer** and **_acceptOffer** to cover both sides of the trade.

MTLX1-14 | Ownership transfer does not revoke the previous owner

Acknowledged

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

./src/libs/BorgAuth.sol

Description:

acceptOwnership() grants **OWNER_ROLE** to the new owner but never revokes it from the previous owner. As a result, both addresses retain **OWNER_ROLE** after the transfer.

The previous owner can therefore continue performing privileged actions such as updating roles, changing role adapters, or initiating another ownership transfer.

```
function initTransferOwnership(address newOwner) external {
    if (newOwner == address(0) || newOwner == msg.sender) revert BorgAuth_ZeroAddress();
    onlyRole(OWNER_ROLE, msg.sender);
    pendingOwner = newOwner;
}

/// @notice accept ownership transfer
function acceptOwnership() external {
    if (msg.sender != pendingOwner) revert BorgAuth_NotAuthorized(OWNER_ROLE, msg.sender);
    _updateRole(pendingOwner, OWNER_ROLE);
    pendingOwner = address(0);
    emit RoleUpdated(pendingOwner, OWNER_ROLE);
}
```

Remediation:

The old owner's role should be revoked when the new owner accepts the transfer.

MTLX1-15 | Certificate Metadata Can Falsely Report the Holder's Balance

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

src/CertificateUriBuilder.sol

Description:

The ledger says the holder owns 1,000 units, while the certificate says 999,999,999. Both are valid JSON, and off-chain consumers may read the forged value.

An issuer can set legalDetails to:

```
{
  "unitsRepresented": "1000.00",       ← the real holding
  "legalDetails": "ok",
  "unitsRepresented": "999999999",     ← added again, and later in the cert displayed
}
```

This produces duplicate **unitsRepresented** fields, with the forged value appearing later in the object. Common JSON parsers use the last value, causing the certificate to report a holding that does not exist.

The value also propagates: **executeSecondaryTransfer** copies **legalDetails** to downstream buyers, allowing the poisoned value to follow the security through subsequent transfers.

Any user can corrupt their own certificate inserting **buyerName** or **EOI.name** without escaping or validation. A user can inject additional fields or make **tokenURI** permanently invalid, breaking consumers that parse the certificate.

Documentation describes the rendered certificate as the stock certificate itself. If that representation is authoritative, allowing it to report a false holding is a material integrity issue.

Root cause: **legalDetails** is inserted into the JSON using **string.concat** without JSON escaping. A quote inside the value can therefore break out of the intended string and inject additional JSON fields.

MTLX1-34 | revokeDeal leaves cancelled certificates active

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

metalex-jul-26/src/storage/DealManagerStorage.sol:revokeDeal#L327-L336

Description:

The **revokeDeal** function lets a party cancel a pending primary deal. When a deal is proposed, its certificates are minted to the **DealManager** up front and held there until the buyer pays and the deal is finalized. **revokeDeal** forwards the party's cancellation request to **CyberAgreementRegistry**, which may immediately void the agreement. This includes the normal case where the issuer cancels while it is still the only signer.

Once the registry marks an agreement as voided, the deal manager should also mark its escrow as **VOIDED** and void the pre-minted certificates. The other cancellation flows, **voidExpiredDeal** and **signToVoid**, already do this through **_voidCorpCerts**. This removes cancelled certificates from the issuer's live ledger and its look-through holder count.

However, **revokeDeal** returns as soon as it calls the registry and does not perform that cleanup:

```
if (LexScrowStorage.getEscrow(agreementId).status == EscrowStatus.PENDING)
    ICyberAgreementRegistry(LexScrowStorage.getDealRegistry())
        .voidContractFor(agreementId, signer, signature);
```

The registry agreement can therefore be voided while the local escrow stays PENDING and its certificates stay active. Those certificates still count as a live position for the manager. If the issuer is at its holder cap, this stale position can make an otherwise valid secondary purchase fail. For an open-counterparty offer, no second named party exists to trigger the missing cleanup, so the certificates remain active until an issuer administrator voids them manually.

An example failure path is:

1. The issuer proposes an open primary deal, which mints certificates to the **DealManager** and creates a pending escrow.
2. The buyer does not pay, so the issuer calls **revokeDeal** while it is the only signer.
3. The registry voids the agreement, but the escrow remains pending and the certificates remain live.
4. A new buyer attempts a secondary purchase while the issuer is at its holder cap. The stale manager position is included in the holder count, causing the purchase to be rejected.

```
    function revokeDeal(bytes32 agreementId, address signer, bytes memory signature) public {
        if (!LexScrowStorage.hasPrimaryEscrow(agreementId)) revert LexScrowStorage.DealDoesNotExist();
        if(msg.sender != signer) revert IDealManagerStorage.CounterPartyValueMismatch();
        if(LexScrowStorage.getEscrow(agreementId).status == EscrowStatus.PENDING)
            ICyberAgreementRegistry(LexScrowStorage.getDealRegistry()).voidContractFor(agreementId, signer,
signature);
        else
            revert IDealManagerStorage.DealNotPending();
    }
```

Remediation:

After calling **voidContractFor**, check whether the registry agreement is voided. If it is, use the same cleanup as **signToVoid** to void the certificates and set a pending escrow to **VOIDED**.

MTLX1-36 | Cross-holder de-scripification leaves ghost holders in the holder count

Fixed

Severity:	Probability:	Impact:
Low	Rare	Low

Path:

metalex-jul-26/src/storage/IssuanceManagerStorage.sol:executeConvertScripToCert#L1127-L1248

Description:

The function **executeConvertScripToCert** converts CyberScrip back into certificate units. If the caller already owns a non-void certificate, the units are added to that certificate. Otherwise, a new certificate is created. Any units not covered by the selected certificate's own vault claim are withdrawn from the shared vault.

Fully scripifying a certificate should not remove its holder from the holder tally straight away. Although its raw units become zero, its vault claim still represents the same effective units. The holder should only be removed once both the raw units and the remaining vault claim reach zero.

However, a cross-holder conversion can remove the last vault claim of the certificate that originally supplied the scrip. The converted units are added to the buyer's certificate, but the now-empty original certificate is not voided or removed from **liveCounted**. This happens because a certificate is considered live based only on whether its status is non-void, not whether it has any effective units left.

The stale entry remains in **lookThroughHolderCount** and **usLookThroughHolderCount**. **HolderCapCondition.checkCondition** trusts these cached values, so it may reject a new buyer even though the real holder count is still below the cap.

An example flow is:

1. Alice fully scripifies her certificate. Its raw balance becomes zero, while its vault claim correctly keeps Alice in the holder tally.
2. Alice transfers the scrip to Bob, who already owns a certificate from the same printer.
3. Bob converts the scrip. The units are added to Bob's certificate, while the shared-vault withdrawal removes Alice's remaining claim.
4. Alice now has no raw or effective units, but her non-void certificate is still counted as a live lot.
5. If the cached count is 100 and the cap is 100, a new buyer is rejected because the condition checks **100 + 1 <= 100**. The real count is only 99, so the purchase should have been allowed.

Remediation:

When a conversion or vault reset removes a certificate's final effective units, void or uncount that certificate and update all cached holder totals. Keep fully scripified certificates counted while they still have a vault claim.

MTLX1-37 | A shared delegate's signature can be replayed for multiple parties

Fixed

Severity:	Probability:	Impact:
Low	Unlikely	Low

Path:

metalex-jul-26/src/CyberAgreementRegistry.sol:signContractFor#L481-L568

Description:

signContractFor lets an agreement party sign directly or through a wallet they previously approved with **setDelegation**. The function builds the agreement's EIP-712 payload, calls **_verifySignature**, and then records the supplied **signer** as having signed. For standalone agreements without a finalizer, any relayer can submit a valid signature and the agreement is finalized once every party is marked as signed.

A delegated signature should only approve the party the delegate intended to represent. Even if two parties use the same operational wallet, a signature made for one of them should not count as consent from the other.

However, **SignatureData** does not include the signer address. **_verifySignature** recovers the delegate from the signed payload, but then looks up the delegation using the unsigned **signer** function argument:

```
function _timestampToDate(uint256 timestamp) private pure returns (string memory) {
    uint256 day = ((timestamp / 86400) % 31) + 1;
    uint256 month = ((timestamp / 2629743) % 12) + 1;
    uint256 year = (timestamp / 31556926) + 1970;
    return string(abi.encodePacked(_uintToString(month), '/', _uintToString(day), '/', _uintToString(year)));
}
```

As a result, the same signature is valid for every party that has approved the recovered wallet, as long as their **partyValues** are identical. A template with no party fields makes those values identical by default. This lets a relayer mark multiple parties as signed even though the delegate signed only once for one party.

An exploit can happen as follows:

1. Parties A and B both delegate signing to wallet D.
2. A standalone agreement is created with A and B as parties and no party fields.
3. D signs the agreement once while acting for A.

4. A relayer calls **signContractFor(A, contractId, [], signature, false, "")**, which records A's signature.
5. The relayer submits the same signature with B as signer. The registry checks B's delegation to D, records B as **signed**, and finalizes the agreement without separate consent for B.

Remediation:

Add the represented **signer** address to **SignatureData** and its EIP-712 type hash, then include it when building the digest. Invalidate the old ambiguous signatures, for example by bumping the EIP-712 domain version.

MTLX1-10 | timestampToDate Returns Incorrect Calendar Dates

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Low

Path:

./src/creds/LeXcheX.sol
./src/CertificateImageBuilder.sol

Description:

```
function _timestampToDate(uint256 timestamp) private pure returns (string memory) {
    uint256 day = ((timestamp / 86400) % 31) + 1;
    uint256 month = ((timestamp / 2629743) % 12) + 1;
    uint256 year = (timestamp / 31556926) + 1970;
    return string(abi.encodePacked(_uintToString(month), '/', _uintToString(day), '/', _uintToString(year)));
}
```

These divisors do not follow the Gregorian calendar, causing the rendered date to become increasingly inaccurate. For example, 8/8/2026 is rendered as 8/29/2026.

A correct **timestampToDate** implementation using **civilFromDays** already exists in src/creds/**LeXcheXBadgeRender.sol**, but these two locations still use the old implementation.

Impact

Expiry enforcement still uses the raw timestamp, so on-chain expiry checks are unaffected.

However, users may see an incorrect expiry date on compliance credentials.

Remediation:

Replace both implementations with the existing **civilFromDays**-based implementation.

MTLX1-11 | Fee is determined at execution, allowing fee changes to affect existing trades

Acknowledged

Severity:	Probability:	Impact:
Informational	Rare	Low

Path:

./src/SecondaryTradeStorage.sol

Description:

The fee for a trade is determined at execution time rather than when the trade is created:

```
uint256 fee = secEscrow.paymentAmount
    * IDealManagerFactory(upgradeFactory).getDefaultFeeRatio()
    / DealManagerFactoryStorage.BASIS_POINTS;
```

This means an existing trade can be affected if the admin changes the default fee before the trade is finalized. For example, a trade created with a 1% fee could later be executed with a 100% fee.

Additionally, **setDefaultFeeRatio()** allows the fee to be set as high as 100%:

```
if (feeRatio > BASIS_POINTS) {
    revert InvalidFeeRatio();
}
```

A lower maximum, such as 10%, would provide better protection for trades.

Remediation:

Consider setting the fee bps in the trade offer.

MTLX1-13 | acceptOwnership() emits an incorrect RoleUpdated event

Acknowledged

Severity:	Probability:	Impact:
Informational	Rare	Low

Path:

./src/libs/BorgAuth.sol

Description:

acceptOwnership() clears **pendingOwner** before emitting the **RoleUpdated** event:

```
pendingOwner = address(0);
emit RoleUpdated(pendingOwner, OWNER_ROLE);
```

As a result, the event is emitted with **address(0)** instead of the new owner's address.

Remediation:

The event should be emitted using the new owner's address by using **msg.sender**.

MTLX1-16 | voidCert allows non-existent certificates to be marked as void

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Low

Path:

src/LedgerEntryToken.sol

Description:

voidCert() does not check whether the certificate exists before marking it as **Void**. This differs from **unvoidCert()**, which explicitly rejects non-existent token IDs.

Because certificate IDs are predictable and assigned using **totalSupply()**, an accidental call to **voidCert()** on a non-existent ID can affect the next certificate that is minted.

For example:

1. An empty printer calls **voidCert(5)**.
2. Certificate **5** is later minted to Alice.
3. The certificate is immediately marked as **Void**.
4. Because voided certificates are excluded from the holder tally, Alice is not counted by **lookThroughHolderCount()**.

Alice is nevertheless a real holder despite the flag: her certificate still carries live **unitsRepresented**, she is the legal owner of record.

This creates a mismatch where the holder count excludes Alice while the market still treats her position as active.

Impact

lookThroughHolderCount() can permanently undercount holders. Since this value is used by **HolderCapCondition** to enforce the holder cap, the system may believe fewer holders exist than actually do.

Re-voiding the certificate does not fix the issue because the certificate was never counted in the first place.

Remediation:

Check that the certificate exists before voiding it:

```
function voidCert(uint256 tokenId) external onlyIssuanceManagerOrAdmin {
    if (!_exists(tokenId)) {
```

```
        revert ILedgerEntryToken.TokenDoesNotExist();
    }

    LedgerEntryTokenStorage.setSecurityStatus(
        tokenId,
        SecurityStatus.Void
    );
    LedgerEntryTokenStorage.recordVoidLegalOwner(tokenId);
}
```

MTLX1-19 | Core component setters emit no events

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Low

Path:

CyberCorp.sol, DealManager.sol

Description:

Several setters update protocol components such as the **IssuanceManager**, **DealManager**, **RoundManager**, **Corp**, and **DealRegistry** without emitting events.

```
function setIssuanceManager(address _issuanceManager) external onlyOwner() {
    issuanceManager = _issuanceManager;
}
```

This makes configuration changes invisible to off-chain indexers and monitoring systems, making it harder to track or detect unauthorized changes.

Remediation:

Emit an event for each component update containing the old and new address.

hexens
x